

UNIVERSITÀ POLITECNICA DELLE MARCHE

Sistema gestionale Azienda Agricola

Progetto di Ingegneria del Software

Giacomo Sabbatini, Gabriele Piersigilli,
Fabio Menghini, Edoardo Marchetti

Anno accademico 2025/2026

Indice

1 Descrizione progetto **4**

2 Glossario **5**

3 Analisi dei requisiti **6**
3.1 Requisiti funzionali 6
3.2 Requisiti non funzionali 8

4 Diagrammi dei casi d'uso **10**
4.1 Diagramma degli attori 10
4.2 Diagramma Gestione Utenti 11
4.3 Diagramma Gestione Entrate 18
4.4 Diagramma Gestione Uscite 23
4.5 Diagramma Gestione Prdotti 28
4.6 Diagramma Gestione Sistema 34
4.7 Matrice di mapping 40

5 Diagrammi di analisi **41**
5.1 Diagramma delle classi di analisi 41
5.1.1 Package di analisi 41
5.1.2 Package di analisi: Utente 42
5.1.3 Package di analisi: Movimento 43
5.1.4 Package di analisi: Prodotto 44
5.1.5 Package di analisi: Sistema 45
5.2 Diagrammi di sequenza 46
5.2.1 AccessoProfiloUtente 46
5.2.2 AggiungiProdotto 47
5.2.3 RegistraSpeseTasse 48
5.2.4 GuadagnoAziendale 49
5.3 Diagrammi di attività 50
5.3.1 Accessi Profilo Utente 50
5.3.2 Aggiungi Prodotto 51
5.3.3 Guadagno Aziendale 52
5.3.4 Registra Entrate 53
5.3.5 Registra Uscite 54

6 Diagrammi di progettazione **55**
6.1 Diagramma delle classi di progettazione 55
6.1.1 Package di progettazione 55
6.1.2 Package di progettazione: Utente 56

6.1.3 Package di progettazione: Movimento 57
6.1.4 Package di progettazione: Prodotto 58
6.1.5 Package di progettazione: Sistema 59
6.2 Diagrammi delle macchine a stati 60
6.2.1 Movimento Finanziario 60
6.2.2 Profilo Utente 61
6.2.3 Sessione Utente 62
6.3 Diagramma dei componenti 63
6.4 Diagramma di entità-relazione 64

7 Implementazione **65**
7.1 Diagramma di deployment 65
7.2 Mockup 66
7.3 Descrizione unit tests 68
7.3.1 Domain test 68
7.3.2 Services tests 69
7.3.3 Persistance tests 77
7.3.4 Views tests 77

1 Descrizione progetto

Il progetto prevede la realizzazione di un sistema gestionale desktop per un'azienda agricola. Il software nasce con l'obiettivo di sostituire la gestione manuale dei conti aziendali, permettendo di registrare in un unico strumento entrate, uscite, prodotti venduti e personale.

Al primo avvio viene registrato il manager, cioè il proprietario dell'azienda, che ha accesso a tutte le funzionalità e può creare, modificare ed eliminare i profili dei dipendenti. I dipendenti accedono con le proprie credenziali e hanno permessi limitati. Ogni utente può modificare il proprio profilo. La password deve avere almeno 8 caratteri alfanumerici, l'email deve essere unica e la sessione scade dopo un periodo di inattività.

Il catalogo raccoglie i prodotti agricoli dell'azienda. Ogni prodotto, identificato da un nome unico, appartiene a una categoria con la propria unità di misura ed è descritto da una descrizione e da un prezzo unitario. Il sistema permette di aggiungere, modificare ed eliminare prodotti e categorie.

Le entrate rappresentano le vendite e sono registrate indicando prodotto, quantità, importo, data e una descrizione di massimo 500 caratteri. Ogni vendita è associata a un cliente, che può essere un'azienda, identificata da ragione sociale, partita IVA e codice SDI, oppure un privato, identificato da nome, cognome e codice fiscale. In questo modo le entrate sono separate per tipo di cliente.

Le uscite comprendono le spese di manutenzione, produzione e vendita, le tasse, gli stipendi, le assicurazioni, lo smaltimento rifiuti e le spese straordinarie, con l'indicazione facoltativa del fornitore e del prodotto di riferimento.

A ogni movimento è possibile allegare uno o più documenti PDF, come fatture o ricevute, visualizzabili direttamente dal programma. Entrate e uscite possono essere modificate, eliminate e filtrate per prodotto, per anno e, nel caso delle entrate, per tipo di cliente.

Nella sezione dedicata al guadagno aziendale, accessibile solo al manager, il sistema calcola per l'anno selezionato il totale delle entrate, il totale delle uscite e il margine netto, e permette di generare un report riepilogativo.

L'applicazione è sviluppata in Python 3, con interfaccia grafica realizzata in PyQt5 e salvataggio dei dati in file JSON.

2 Glossario

Termine	**Descrizione**	**Tipo**	**Sinonimi**
Utente	Individuo che usufruisce del programma	Tecnico	Utilizzatore
Entrata	Flusso di denaro che entra nel bilancio dell'azienda	Business	Ricavo, Incasso
Uscita	Flusso di denaro che esce dal bilancio dell'azienda	Business	Perdita, Spesa, Costo
Backup	Creazione di una copia di sicurezza di dati, file o programmi da conservare	Tecnico	Duplicato digitale
Profilo	Insieme di informazioni, dati e caratteristiche che identificano una persona nell'azienda	Tecnico	Identità virtuale/digitale
Credenziali	Combinazione di nome+password unica che identifica ogni utilizzatore del programma e che dimostra il diritto di usare un determinato servizio	Tecnico	Referenze, Dati di accesso
Dipendente	Sottocategoria dell'Utente, riguarda colui che può utilizzare il programma ma sotto l'autorità di un titolare	Tecnico	Nessuno
Manager	Sottocategoria dell'Utente, riguarda colui che è proprietario dell'azienda e ha accesso a tutti i servizi del programma	Tecnico	Capo, Datore
Accesso	Il riconoscimento delle credenziali per entrare in un sistema o account	Tecnico	Login, Autenticazione, Entrata

3 Analisi dei requisiti

3.1 Requisiti funzionali

Area: Gestione Utenti

RF1: Registrazione profilo Manager
Il sistema dovrà consentire al manager di registrarsi.

RF2: Creazione profilo dipendente
Il sistema dovrà consentire al manager di creare un profilo per un dipendente.

RF3: Accesso profilo utente
Il sistema dovrà consentire agli utenti di accedere all'account.

RF4: Modifica profilo utente e manager
Il sistema dovrà consentire agli utenti di gestire il proprio profilo, e al manager di modificare il profilo dei dipendenti.

RF5: Elimina profilo dipendente
Il sistema dovrà consentire ai manager di eliminare il profilo di un dipendente.

Area: Gestione prodotti

RF6: Aggiungi categoria prodotto
Il sistema deve permettere di aggiungere una nuova categoria prodotto.

RF7: Aggiungi prodotto
Il sistema deve permettere di aggiungere un nuovo prodotto agricolo.

RF8: Modifica prodotto
Il sistema deve permettere di modificare un prodotto agricolo presente.

RF9: Elimina prodotto
Il sistema deve permettere di eliminare un prodotto agricolo.

RF10: Elimina categoria prodotto
Il sistema deve permettere di eliminare una categoria prodotto.

Area: Gestione entrate

RF11: Registrazione entrate per prodotto
Il sistema deve permettere la registrazione delle vendite catalogate per prodotti.

RF12: Separazione delle entrate in base al tipo di cliente
Il sistema deve permettere di registrare l'entrata distinguendola in base al

tipo di cliente.

RF13: Modifica entrata
Il sistema deve permettere di modificare un'entrata selezionata.

RF14: Elimina entrata
Il sistema deve permettere di eliminare un'entrata selezionata.

Area: Gestione uscite

RF15: Registrazione spese di manutenzione
Il sistema deve permettere la registrazione di spese legate alla manutenzione di varia natura.

RF16: Registrazione spese di produzione
Il sistema deve permettere la registrazione di spese legate alla produzione dei prodotti agricoli.

RF17: Registrazione spese di vendita
Il sistema deve permettere la registrazione di spese di confezionamento.

RF18: Registrazione delle spese dovute a tasse
Il sistema deve permettere la registrazione delle spese dovute alle tasse.

RF19: Registrazione del pagamento dei salari dei dipendenti
Il sistema deve permettere la registrazione del pagamento dei salari dei dipendenti.

RF20: Registrazione delle spese dovute ad assicurazioni
Il sistema deve permettere la registrazione delle spese derivanti dal pagamento di assicurazioni.

RF21: Registrazione delle spese straordinarie
Il sistema deve permettere la registrazione di tutte le spese che non rientrano nelle categorie precedenti.

RF22: Separazione delle uscite in base al tipo di fornitore
Il sistema deve permettere di registrare l'uscita distinguendola in base al tipo di fornitore.

RF23: Modifica uscita
Il sistema deve permettere di modificare un'uscita selezionata.

RF24: Elimina uscita
Il sistema deve permettere di eliminare un'uscita selezionata.

Area: Gestione sistema

RF25: Allegazione di documento relativo al movimento finanziario
Il sistema deve permettere di allegare uno o più documenti relativi alle entrate ed alle uscite, in più deve permettere di visualizzare tale documento.

RF26: Classificazione delle entrate
Il sistema deve mostrare le entrate catalogate per prodotto, per anno e per cliente.

RF27: Classificazione delle uscite
Il sistema deve mostrare le uscite catalogate per prodotto, per anno e per cliente.

RF28: Guadagno aziendale
Il sistema deve mostrare in una sezione guadagno la differenza fra entrate ed uscite.

RF29: Backup
Il sistema dovrà effettuare un backup dei dati su file JSON ogni volta che viene modificata un cliente, un prodotto, un utente, un movimento.

3.2 Requisiti non funzionali

RNF1: Implementazione in Python 3
Il sistema dovrà essere implementato in linguaggio Python 3.

RNF2: Implementazione grafica in PyQt5
Il sistema dovrà implementare un'interfaccia grafica in PyQt5.

RNF3: Validazione password
Il sistema dovrà validare la password di un utente o un owner solo se è di almeno 8 caratteri alfanumerici.

RNF4: Unicità email
Il sistema non dovrà consentire l'utilizzo di un'email utente già in uso.

RNF5: Unicità prodotto e categoria prodotto
Il sistema non dovrà consentire l'aggiunta di un prodotto o di una categoria prodotto già esistente.

RNF6: Gestione data inserimento prodotto
Il sistema dovrà permettere di inserire la data di vendita e acquisto del prodotto.

RNF7: Limite di caratteri della descrizione
Il sistema non dovrà accettare descrizioni con più di 500 caratteri.

4 Diagrammi dei casi d'uso

4.1 Diagramma degli attori

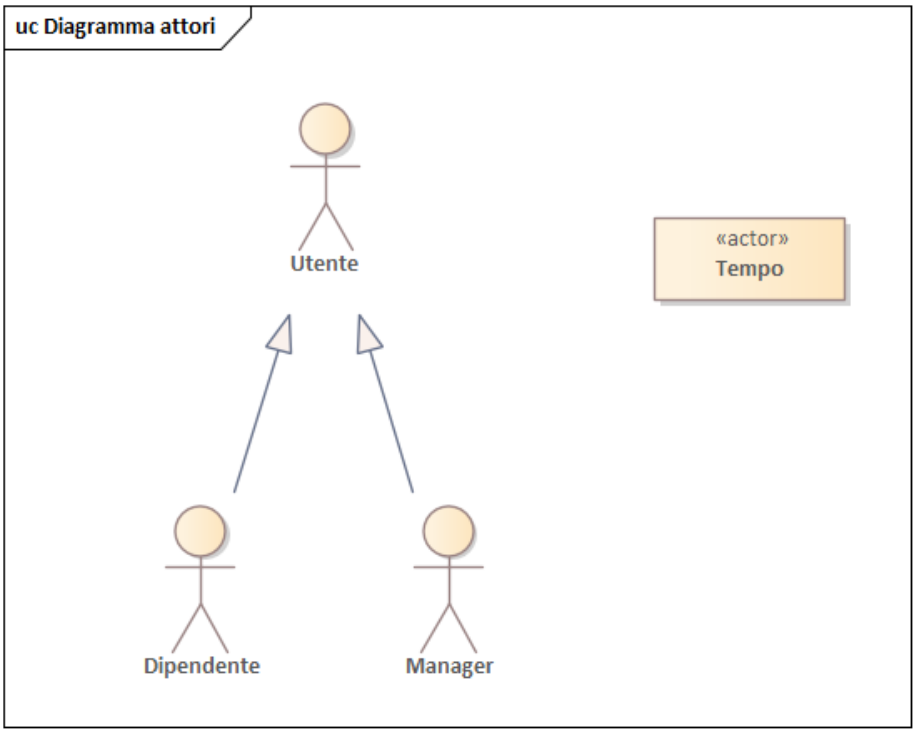


Figura 1: Diagramma degli Attori

4.2 Diagramma Gestione Utenti

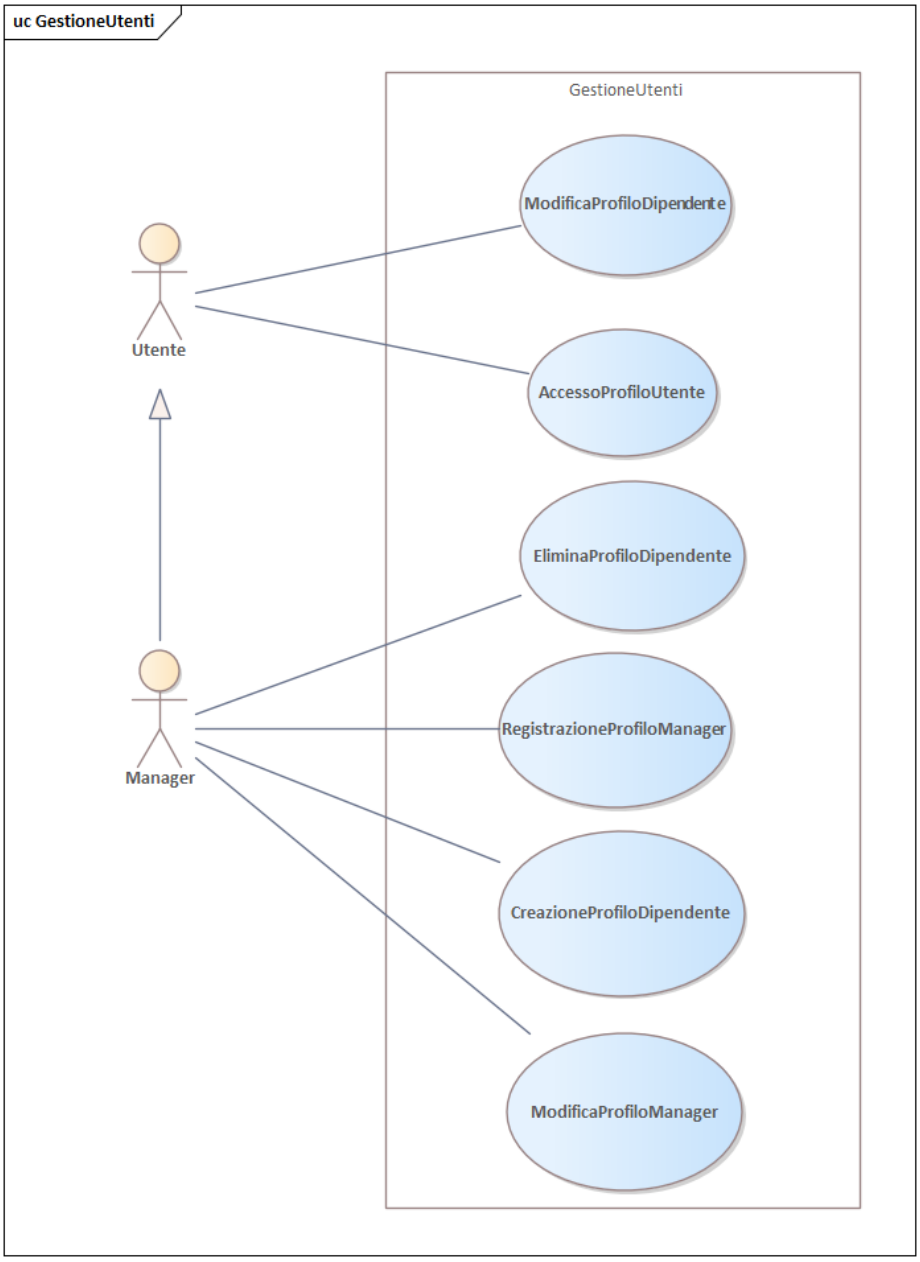


Figura 2: Diagramma Gestione Utenti

Caso d'uso: ModificaProfiloUtente

Caso d'uso: ModificaProfiloUtente
ID: 1
Breve Descrizione: Permette all'Utente di modificare il proprio profilo
Attore: Utente
Precondizioni: Il profilo deve essere già esistente
Sequenza degli eventi principale: 1. L'Utente, dalla schermata principale, modifica i dati del profilo 2. Il sistema salva i dati modificati
Postcondizioni: 1. Una volta confermata la modifica il sistema non può tornare indietro con i dati salvati prima dell'operazione, bensì può permettere di modificare ulteriormente lo stesso profilo
Sequenza degli eventi alternativa: 1. L'Utente non inserisce sufficienti dati e/o inserisce dati non validi. Il sistema riavvia l'operazione informando l'utente dell'errore 2. Il programma presenta un errore durante l'elaborazione dei dati. Il sistema informa l'utente dell'errore e ritorna alla schermata di inserimento dati

Caso d'uso:AccessoProfiloUtente

Caso d'uso: AccessoProfiloUtente
ID: 2
Breve Descrizione: Permette all'Utente di accedere al proprio profilo Dipendente
Attore: Utente
Precondizioni: Il profilo deve essere già esistente
Sequenza degli eventi principale: 1. L'Utente avvia il programma 2. Il sistema chiede di confermare l'identità con l'inserimento delle credenziali del Profilo 3. L'Utente inserisce le credenziali 4. Il programma reinderizza l'Utente alla schermata principale del profilo
Postcondizioni: Nessuna
Sequenza degli eventi alternativa: 1. Il programma presenta un errore durante l'avviamento. Il sistema informa l'utente dell'errore e chiude il programma 2. Il programma presenta un errore durante l'inserimento delle credenziali. Il sistema informa l'utente dell'errore e ritorna alla schermata di accesso

Caso d'uso: EliminaProfiloDipendente

Caso d'uso: EliminaProfiloDipendente
ID: 3
Breve Descrizione: Permette al Manager di cancellare definitivamente il profilo di un Dipendente
Attore: Manager
Precondizioni: Il profilo deve essere già esistente
Sequenza degli eventi principale: 1. Il Manager seleziona il profilo da Modificare 2. Il Manager seleziona "Elimina Profilo Dipendente" 3. Il programma chiede la conferma dell'eliminazione 4. Il Manager conferma l'eliminazione 5. Il profilo Dipendente viene eliminato
Postcondizioni: 1. Il profilo Dipendente eliminato non esisterà più
Sequenza degli eventi alternativa: 1. Il programma presenta un errore durante l'eliminazione. Il sistema riavvia l'operazione informando l'utente dell'errore

Caso d'uso: RegistrazioneProfiloManager

Caso d'uso: RegistrazioneProfiloManager
ID: 4
Breve Descrizione: Permette al Manager di registrarsi per la prima volta
Attore: Manager
Precondizioni: Le credenziali iniziali vengono fornite dagli operatori del programma
Sequenza degli eventi principale: 1. Il Manager avvia il programma 2. Il programma chiede di confermare l'identità con l'inserimento delle credenziali del Profilo 3. Il Manager inserisce le credenziali fornite 4. Il programma esegue la registrazione e reinderizza il Manager all'interno della schermata principale del profilo
Postcondizioni: 1. Il profilo Manager è registrato con la possibilità di modificare le credenziali d'accesso
Sequenza degli eventi alternativa: 1. Il programma presenta un errore durante l'avviamento. Il sistema informa l'utente dell'errore e chiude il programma 2. Il programma presenta un errore durante l'inserimento delle credenziali. Il sistema informa l'utente dell'errore e ritorna alla schermata di accesso

Caso d'uso: CreazioneProfiloDipendente

Caso d'uso: CreazioneProfiloDipendente
ID: 5
Breve Descrizione: Permette al Manager di creare un nuovo profilo Dipendente
Attore: Manager
Precondizioni: Nessuna
Sequenza degli eventi principale: 1. Il programma chiede l'inserimento dei dati del nuovo Dipendente 2. Il Manager inserisce i dati richiesti 3. Il programma esegue la creazione del nuovo profilo
Postcondizioni: Nessuna
Sequenza degli eventi alternativa: 1. Il programma presenta un errore durante l'inserimento dei dati. Il sistema informa l'utente dell'errore e ritorna alla schermata di inserimento 2. Il sistema presenta un errore durante la creazione del nuovo profilo. Il sistema informa l'utente dell'errore e ritorna alla schermata principale

Caso d'uso: ModificaProfiloManager

Caso d'uso: ModificaProfiloManager
ID: 6
Breve Descrizione: Permette al Manager di modificare il profilo di un Dipendente
Attore: Manager
Precondizioni: Il profilo deve essere già esistente
Sequenza degli eventi principale: 1. Il Manager seleziona un profilo Dipendente 2. Il manager seleziona "Modifica Profilo" 3. Il manager inserisce i nuovi dati 4. Il programma esegue la modifica del profilo
Postcondizioni: 1. Una volta confermata la modifica il sistema non può tornare indietro con i dati salvati prima dell'operazione, bensì può permettere di modificare ulteriormente lo stesso profilo
Sequenza degli eventi alternativa: 1. L'Utente non inserisce sufficienti dati e/o inserisce dati non validi. Il sistema riavvia l'operazione informando l'utente dell'errore 2. Il programma presenta un errore durante l'elaborazione dei dati. Il sistema informa l'utente dell'errore e ritorna alla schermata di inserimento dati

4.3 Diagramma Gestione Entrate

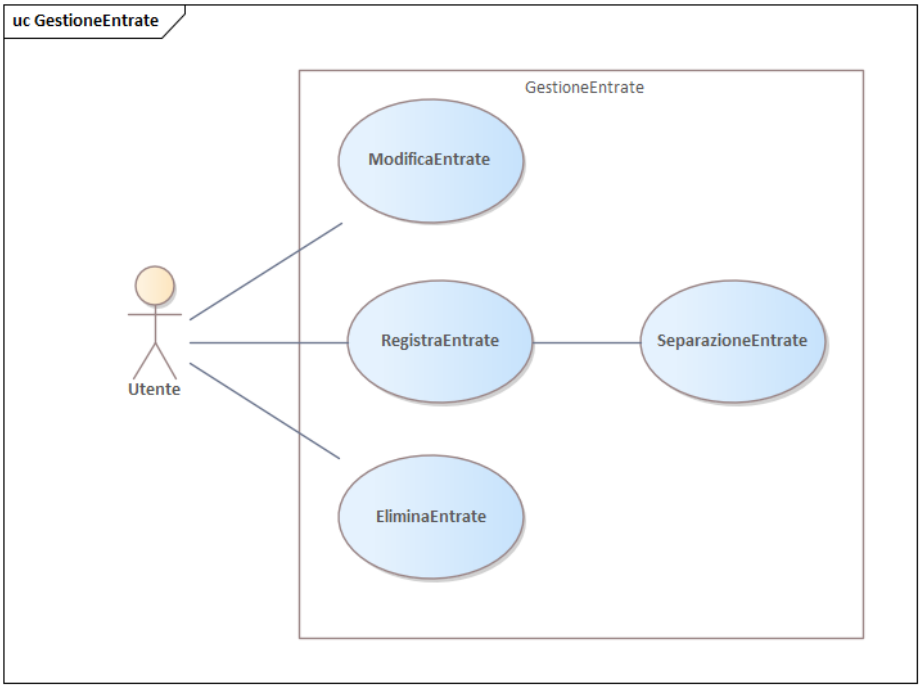


Figura 3: Diagramma Gestione Entrate

Caso d'uso: RegistraEntrate

Caso d'uso: RegistraEntrate
ID: 7
Breve Descrizione: Permette all'utente di registrare le entrate
Attore: Utente
Precondizioni: Nessuna
Sequenza degli eventi principale: 1. L'utente seleziona "Registra Nuova Entrata" 2. Il sistema mostra il modulo per l'inserimento dei dati 3. L'utente inserisce i dati 4. L'utente conferma l'invio dei dati 5. Il sistema verifica la correttezza dei dati 6. Il sistema registra i dati
Postcondizioni: 1. L'entrata è registrata nel programma e disponibile ulteriori successive elaborazioni
Sequenza degli eventi alternativa: 1. L'utente non inserisce sufficienti dati e/o inserisce dati non validi. Il sistema riavvia l'operazione informando l'utente dell'errore 2. Il programma presenta un errore durante il caricamento dei dati. Il sistema informa l'utente dell'errore e ritorna alla schermata di inserimento dati

Caso d'uso: ModificaEntrata

Caso d'uso: ModificaEntrata
ID: 8
Breve Descrizione: Permette all'Utente di modificare le entrate già registrate
Attore: Utente
Precondizioni: 1. L'entrata deve essere già stata registrata
Sequenza degli eventi principale: 1. L'Utente seleziona l'entrata da modificare 2. L'Utente seleziona "Modifica Entrata" 3. L'Utente modifica i dati dell'entrata 4. Il sistema registra i dati
Postcondizioni: 1. Una volta confermata la modifica il sistema non può tornare indietro all'entrata prima della modifica, bensì può permettere di modificare ulteriormente la stessa entrata
Sequenza degli eventi alternativa: 1. Il sistema presenta un problema durante l'elaborazione dei dati. Il sistema informa l'utente dell'errore e riavvia l'operazione

Caso d'uso: EliminaEntrate

Caso d'uso: EliminaEntrate
ID: 9
Breve Descrizione: Permette all'Utente di eliminare un'entrata per sempre
Attore: Utente
Precondizioni: 1. L'entrata deve essere già esistente
Sequenza degli eventi principale: 1. L'Utente seleziona l'entrata da eliminare 2. L'Utente clicca "Elimina Entrata" 3. L'entrata viene eliminata
Postcondizioni: 1. L'entrata non sarà più presente nel software 2. L'unico modo per riavere l'entrata è inserirla nuovamente a mano
Sequenza degli eventi alternativa: 1. Il sistema presenta un problema durante l'eliminazione dell'entrata. Il sistema informa l'utente dell'errore e riavvia l'operazione

Caso d'uso: SeparazioneEntrate

Caso d'uso: SeparazioneEntrate
ID: 10
Breve Descrizione: Permette all'utente di separare le entrate in base al tipo di cliente o al tipo di prodotto venduto
Attore: Utente
Precondizioni: Nessuna
Sequenza degli eventi principale: 1. L'Utente seleziona "Registra Nuova Entrata" 2. Durante la registrazione dell'entrata l'utente può selezionare il tipo di cliente (Azienda o Privato)
Postcondizioni: Nessuna
Sequenza degli eventi alternativa: Nessuna

4.4 Diagramma Gestione Uscite

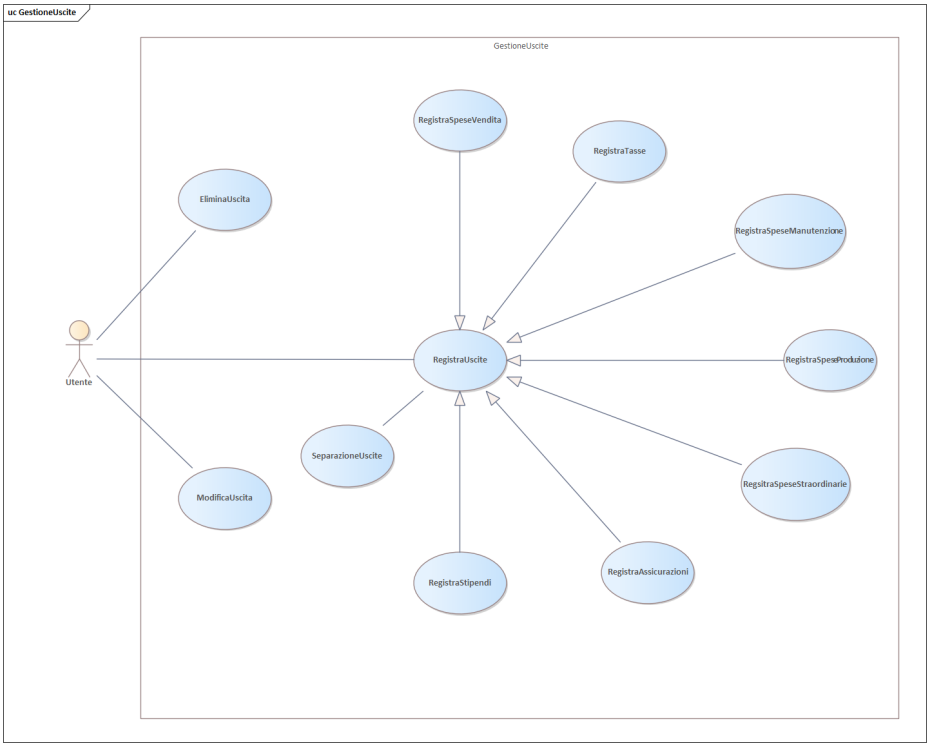


Figura 4: Diagramma Gestione Uscite

Caso d'uso: RegistraUscite

Caso d'uso: RegistraUscite
ID: 11
Breve Descrizione: Permette all'utente di registrare le uscite
Attore: Utente
Precondizioni: Nessuna
Sequenza degli eventi principale: 1. L'Utente seleziona "Nuova Uscita" 2. Il sistema mostra il modulo per l'inserimento dei dati 3. L'Utente seleziona la categoria (...) 4. L'Utente gli altri dati richiesti 5. L'Utente conferma l'invio dei dati 6. Il sistema verifica la correttezza dei dati 7. Il sistema registra i dati
Postcondizioni: 1. L'uscita è registrata nel programma e disponibile ulteriori successive elaborazioni
Sequenza degli eventi alternativa: 1. L'Utente non inserisce sufficienti dati e/o inserisce dati non validi. Il sistema riavvia l'operazione informando l'utente dell'errore 2. Il programma presenta un errore durante il caricamento dei dati. Il sistema informa l'utente dell'errore
Note: RegistraSpeseVendita, RegistraTasse, RegistraSpeseManutenzione, RegistraSpeseProduzione, RegistraSpeseStraordinarie, RegistraAssicurazioni e RegistraStipendi rappresentano specializzazioni del caso d'uso RegistraUscite. L'unica differenza consiste nella categoria in cui l'entrata viene memorizzata (spese vendita, tasse, spese manutenzione, spese produzione, spese straordinarie, assicurazioni e stipendi), mentre il flusso operativo è identico.

Caso d'uso: ModificaUscita

Caso d'uso: ModificaUscita
ID: 12
Breve descrizione: Permette all'utente di modificare le uscite già registrate
Attore: Utente
Precondizioni: 1. L'uscita da modificare deve essere già stata registrata
Sequenza degli eventi principale: 1. L'Utente seleziona l'uscita da modificare 2. L'Utente seleziona "Modifica Uscita" 3. L'Utente modifica i dati dell'uscita 4. Il sistema registra i dati
Postcondizioni: 1. Una volta confermata la modifica il sistema non può tornare indietro all'entrata prima della modifica, bensì può permettere di modificare ulteriormente la stessa uscita
Sequenza degli eventi alternativa: 1. Il sistema presenta un problema durante l'elaborazione dei dati. Il sistema informa l'utente dell'errore e riavvia l'operazione

Caso d'uso: EliminaUscita

Caso d'uso: EliminaUscita
ID: 13
Breve Descrizione: Permette all'Utente di eliminare un'uscita per sempre
Attore: Utente
Precondizioni: 1. L'Uscita deve essere già esistente
Sequenza degli eventi principale: 1. L'utente seleziona l'uscita da eliminare 2. L'utente seleziona "Elimina Uscita" 3. L'uscita viene eliminata
Postcondizioni: 1. L'uscita non sarà più presente nel software 2. L'unico modo per riavere l'uscita è inserirla nuovamente a mano
Sequenza degli eventi alternativa: 1. Il sistema presenta un problema durante l'eliminazione dell'uscita. Il sistema informa l'utente dell'errore e riavvia l'operazione

Caso d'uso: SeparazioneUscite

Caso d'uso: SeparazioneUscite
ID: 14
Breve Descrizione: Permette all'utente di separare le uscite in base al tipo di prodotto venduto
Attore: Utente
Precondizioni: Nessuna
Sequenza degli eventi principale: 1. L'Utente seleziona "Registra Nuova Uscita" 2. Durante la registrazione dell'uscita l'utente può selezionare il tipo di prodotto
Postcondizioni: Nessuna
Sequenza degli eventi alternativa: Nessuna

4.5 Diagramma Gestione Prdotti

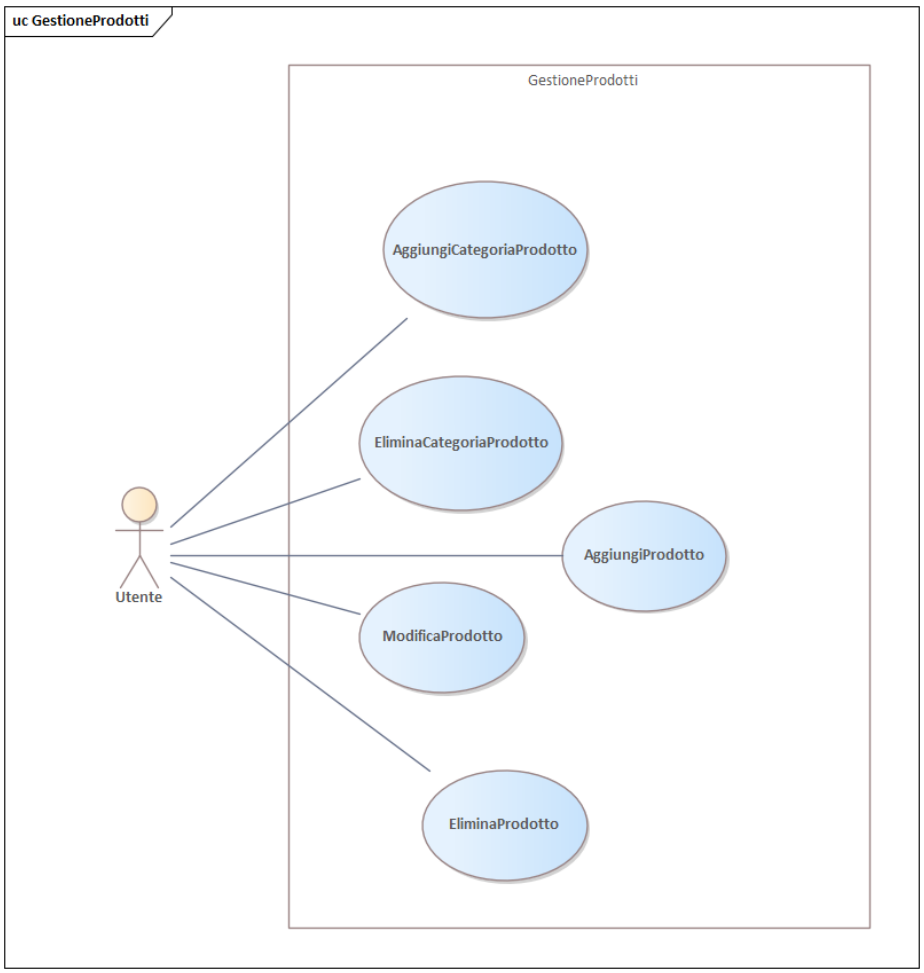


Figura 5: Diagramma Gestione Prodotti

Caso d'uso: AggiungiCategoriaProdotto

Caso d'uso: AggiungiCategoriaProdotto
ID: 15
Breve Descrizione: Permette all'utente di aggiungere una categoria che apparirà durante la registrazione di una nuova entrata o uscita
Attore: Utente
Precondizioni: Nessuna
Sequenza degli eventi principale: 1. L'Utente seleziona "Aggiungi Nuova Categoria" 2. L'Utente inserisce il nome e l'unità di misura del prodotto 3. Il sistema registra la nuova categoria
Postcondizioni: 1. Durante la registrazione di un movimento l'Utente potrà scegliere una categoria nuova
Sequenza degli eventi alternativa: 1. Il sistema presenta un errore durante l'aggiunta della categoria. Il sistema informa l'utente dell'errore e riavvia l'operazione

Caso d'uso: EliminaCategoriaProdotto

Caso d'uso: EliminaCategoriaProdotto
ID: 16
Breve Descrizione: Permette all'utente di eliminare una categoria di prodotti
Attore: Utente
Precondizioni: 1. La categoria deve essere esistente
Sequenza degli eventi principale: 1. L'Utente seleziona "Elimina Categoria Prodotto" 2. L'Utente seleziona la categoria da eliminare 3. L'Utente conferma l'operazione
Postcondizioni: 1. I prodotti associati a quella categoria saranno eliminati automaticamente
Sequenza degli eventi alternativa: 1. Il sistema presenta un errore durante la cancellazione della categoria selezionata. Il sistema informa l'utente dell'errore e riavvia l'operazione

Caso d'uso: AggiungiProdotto

Caso d'uso: AggiungiProdotto
ID: 17
Breve Descrizione: Permette all'Utente di aggiungere un prodotto
Attore: Utente
Precondizioni: Nessuna
Sequenza degli eventi principale: 1. L'Utente seleziona "Aggiungi Prodotto" 2. L'Utente seleziona tutte le caratteristiche relative ad esso 3. Il sistema salva le informazioni
Postcondizioni: 1. Il prodotto potrà essere usato per la registrazione di movimenti futuri
Sequenza degli eventi alternativa: 1. Il sistema presenta un problema durante il salvataggio del prodotto. Il sistema informa l'utente dell'errore e riavvia l'operazione

Caso d'uso: ModificaProdotto

Caso d'uso: ModificaProdotto
ID: 18
Breve Descrizione: Permette all'Utente di modificare un prodotto
Attore: Utente
Precondizioni: 1. Il prodotto deve essere già esistente
Sequenza degli eventi principale: 1. L'Utente seleziona "Modifica Prodotto" 2. L'Utente modifica il prodotto 3. Il sistema salva le modifiche
Postcondizioni: 1. Le caratteristiche del prodotto prima della modifica non possono più essere riportate indietro, a meno che il prodotto non venga registrato a mano come prima
Sequenza degli eventi alternativa: 1. Il sistema presenta un errore durante il salvataggio delle modifiche. Il sistema informa l'utente dell'errore e riavvia l'operazione

Caso d'uso: EliminaProdotto

Caso d'uso: EliminaProdotto
ID: 19
Breve Descrizione: Permette all'utente di eliminare un prodotto
Attore: Utente
Precondizioni: 1. Il prodotto deve essere già esistente
Sequenza degli eventi principale: 1. L'Utente seleziona il prodotto da eliminare 2. L'Utente seleziona "Elimina Prodotto" 3. L'Utente conferma l'eliminazione 4. Il sistema elimina il prodotto
Postcondizioni: 1. Il prodotto non sarà più presente nel software 2. L'unico modo per aver nuovamente quel prodotto sarà inserirlo a mano
Sequenza degli eventi alternativa: 1. Il sistema presenta un errore durante l'eliminazione del prodotto. Il sistema informa l'utente dell'errore e riavvia l'operazione

4.6 Diagramma Gestione Sistema

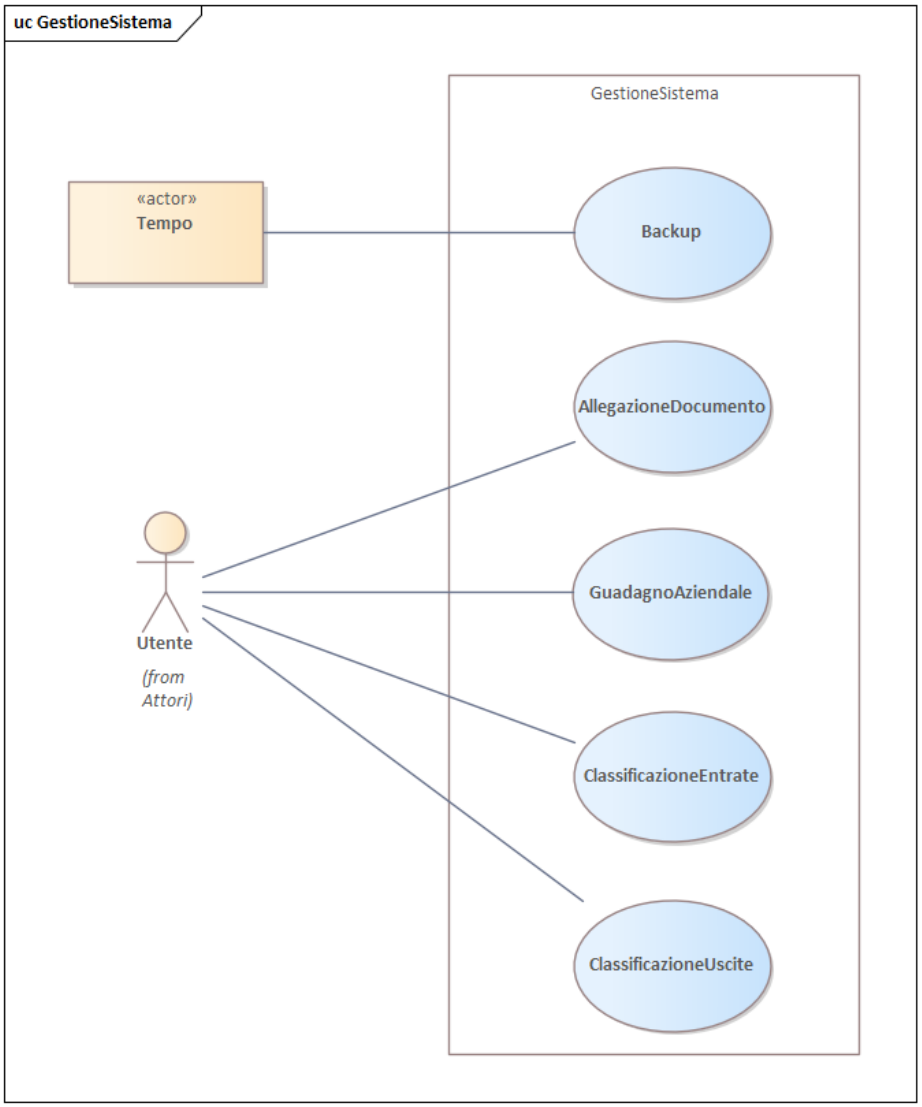


Figura 6: Diagramma Gestione Sistema

Caso d'uso: GuadagnoAziendale

Caso d'uso: GuadagnoAziendale
ID: 20
Breve Descrizione: Permette all'Utente di controllare il guadagno attuale dell'azienda
Attore: Utente
Precondizioni: Nessuna
Sequenza degli eventi principale: 1. L'Utente và nella schermata del backup 2. Il sistema dirà a quanto ammonta il guadagno dell'azienda
Postcondizioni: Nessuna
Sequenza degli eventi alternativa: 1. Il programma presenta un errore durante l'operazione. Il sistema informa l'utente dell'errore

Caso d'uso: Backup

Caso d'uso: Backup
ID: 21
Breve Descrizione: Permette di salvare i dati registrati impedendo la loro perdita
Attore: Tempo
Precondizioni: Nessuna
Sequenza degli eventi principale: 1. Il sistema effettua un backup dei dati su file JSON ogni volta che viene modificata un cliente, un prodotto, un utente, un movimento.
Postcondizioni: Nessuna
Sequenza degli eventi alternativa: 1. Il programma durante il salvataggio dei dati presenta un errore. Il sistema informa l'utente dell'errore e riavvia l'operazione

Caso d'uso: AllegazioneDocumento

Caso d'uso: AllegazioneDocumento
ID: 22
Breve Descrizione: Permette all'Utente di allegare un documento relativo ad un movimento
Attore: Utente
Precondizioni: Nessuna
Sequenza degli eventi principale: 1. L'Utente durante la registrazione dei movimenti seleziona "Allega Documento" 2. Il sistema permette all'Utente di inserire un documento relativo al movimento
Postcondizioni: Nessuna
Sequenza degli eventi alternativa: 1. Il sistema presenta un errore durante il salvataggio del documento. Il sistema informa l'utente dell'errore e riavvia l'operazione

Caso d'uso: ClassificazioneEntrate

Caso d'uso: ClassificazioneEntrate
ID: 23
Breve Descrizione: Permette al sistema di classificare le entrate in base al prodotto, in base all'anno e in base al tipo di cliente
Attore: Utente
Precondizioni: Nessuna
Sequenza degli eventi principale: 1. L'Utente seleziona un prodotto, un anno e/o un tipo di cliente dal relativo menù a tendina 2. Il sistema filtra le entrate in corrispondenza alle selezioni 3. Il sistema mostra le entrate filtrate
Postcondizioni: Nessuna
Sequenza degli eventi alternativa: 1. Il sistema presenta un errore durante la scelta dei filtri. Il sistema informa l'utente dell'errore e riavvia l'operazione

Caso d'uso: ClassificazioneUscite

Caso d'uso: ClassificazioneUscite
ID: 24
Breve Descrizione: Permette al sistema di classificare le uscite in base al tipo di spesa e in base all'anno
Attore: Utente
Precondizioni: Nessuna
Sequenza degli eventi principale: 1. L'Utente seleziona un tipo di spesa e/o un anno dal relativo menù a tendina 2. Il sistema filtra le uscite in corrispondenza alle selezioni 3. Il sistema mostra le uscite filtrate
Postcondizioni: Nessuna
Sequenza degli eventi alternativa: 1. Il sistema presenta un errore durante la scelta dei filtri. Il sistema informa l'utente dell'errore e riavvia l'operazione

4.7 Matrice di mapping

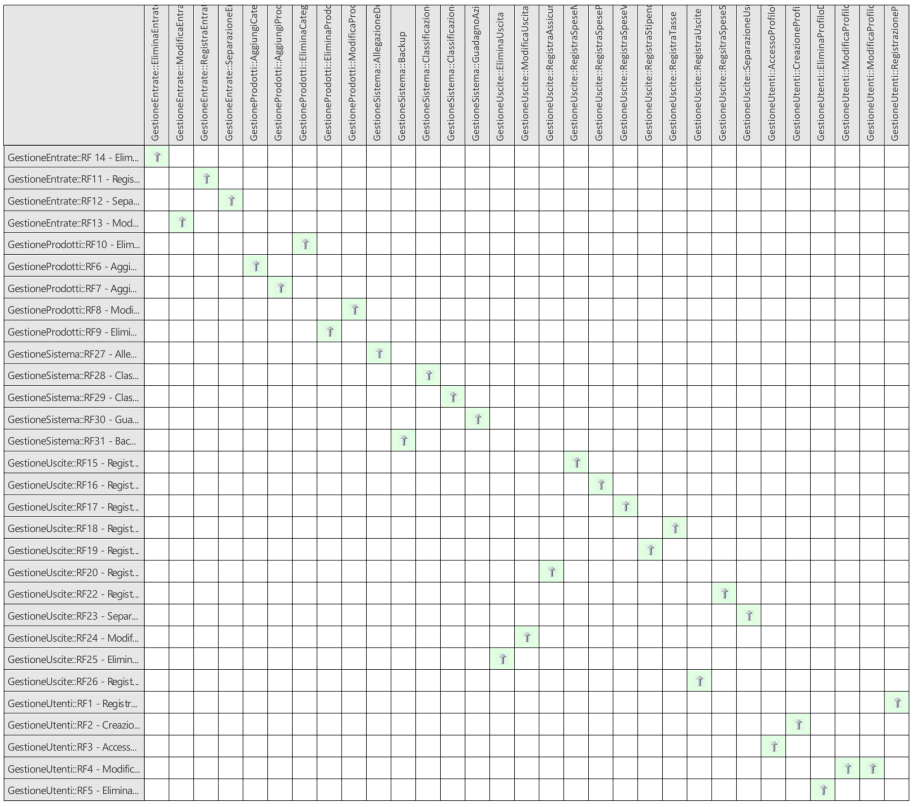


Figura 7: Matrice di mapping

5 Diagrammi di analisi

5.1 Diagramma delle classi di analisi

5.1.1 Package di analisi

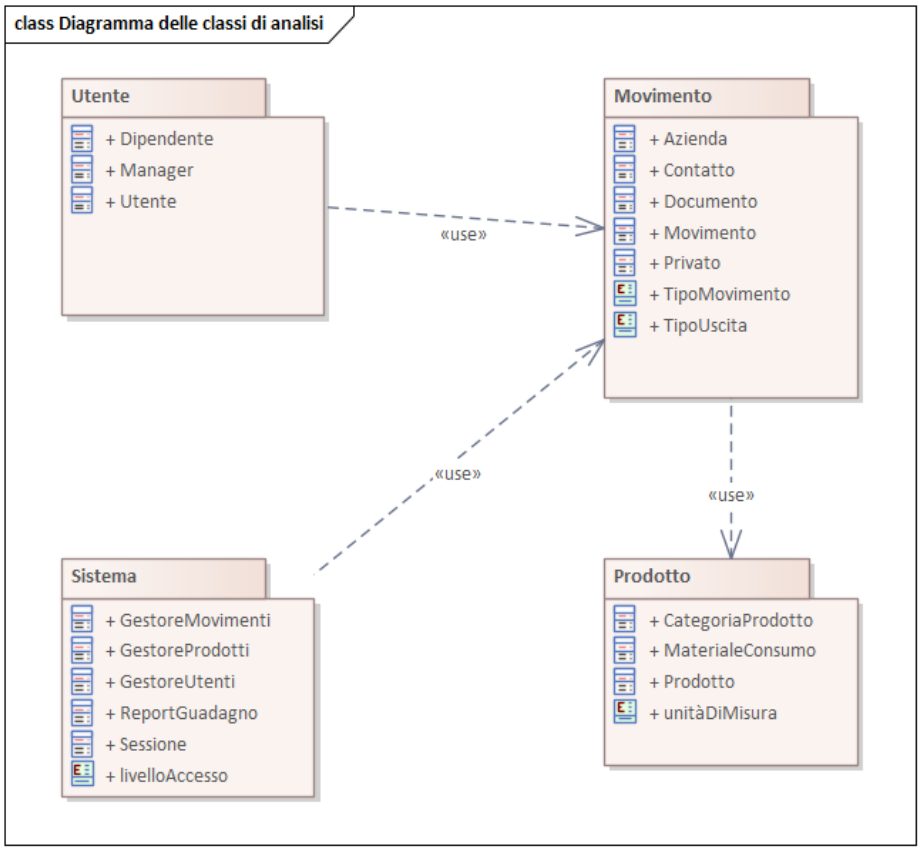


Figura 8: Diagramma delle classi di analisi - Visione d'insieme

5.1.2 Package di analisi: Utente

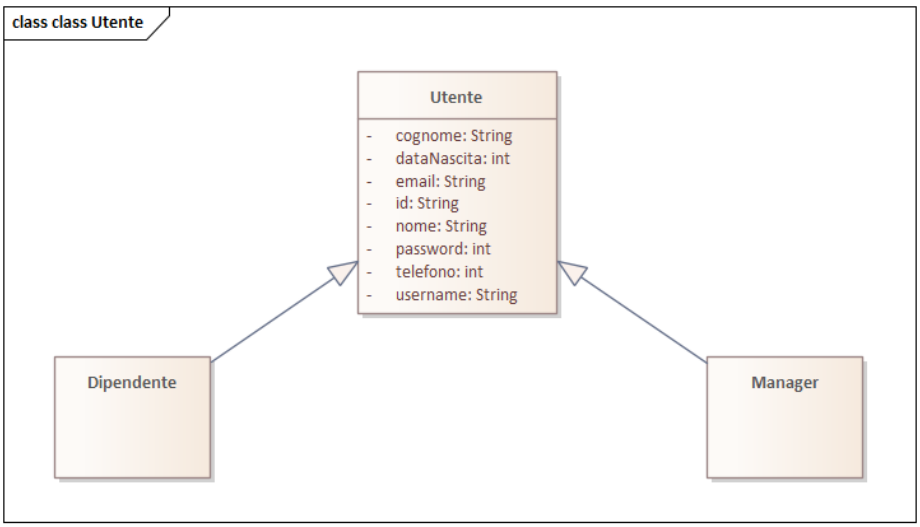


Figura 9: Diagramma delle classi di analisi - Package Utente

5.1.3 Package di analisi: Movimento

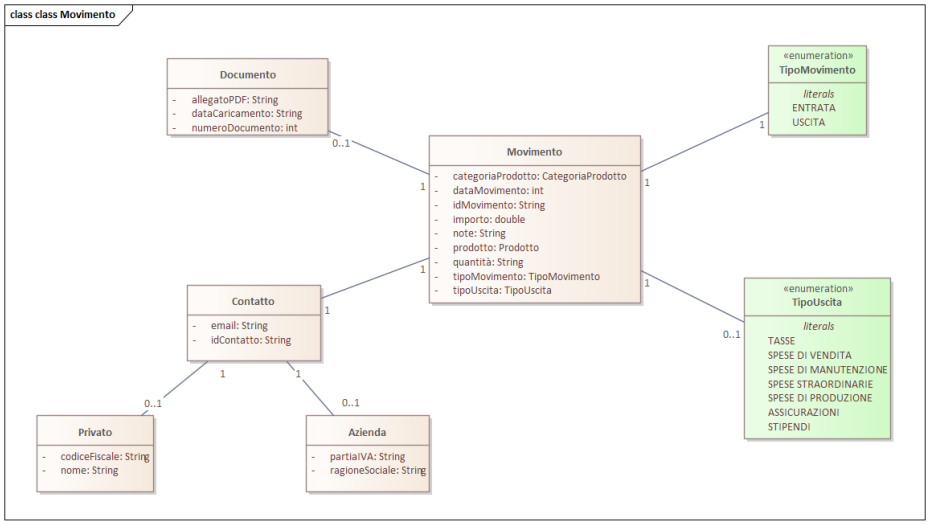


Figura 10: Diagramma delle classi di analisi - Package Movimento

5.1.4 Package di analisi: Prodotto

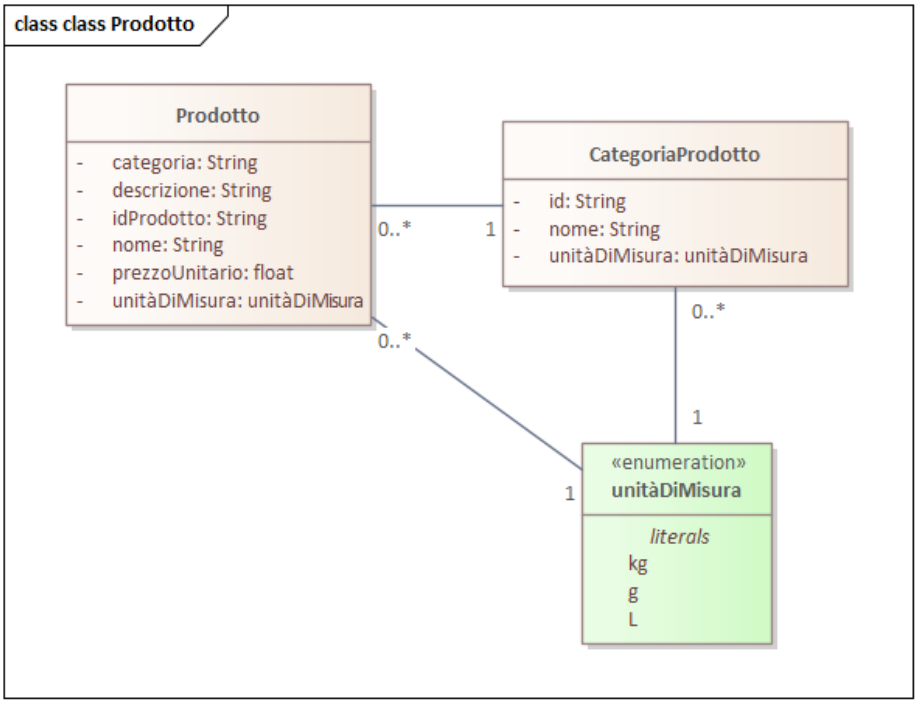


Figura 11: Diagramma delle classi di analisi - Package Prodotto

5.1.5 Package di analisi: Sistema

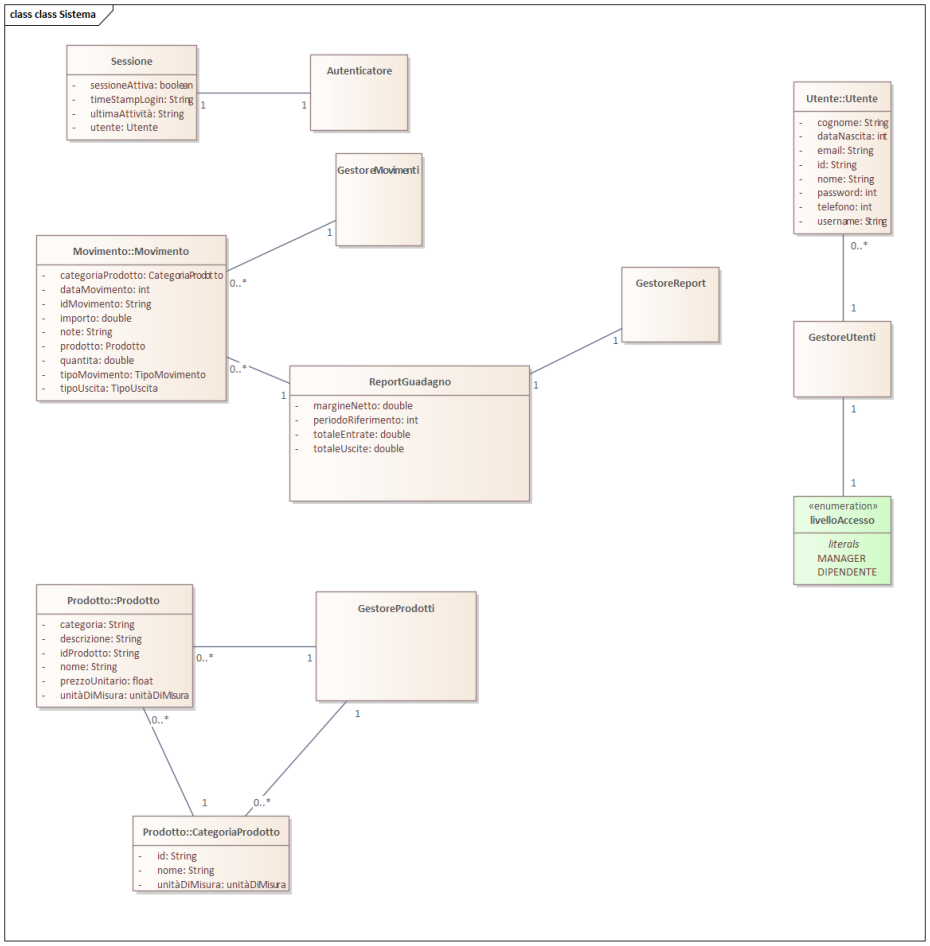


Figura 12: Diagramma delle classi di analisi - Package Sistema

5.2 Diagrammi di sequenza

5.2.1 AccessoProfiloUtente

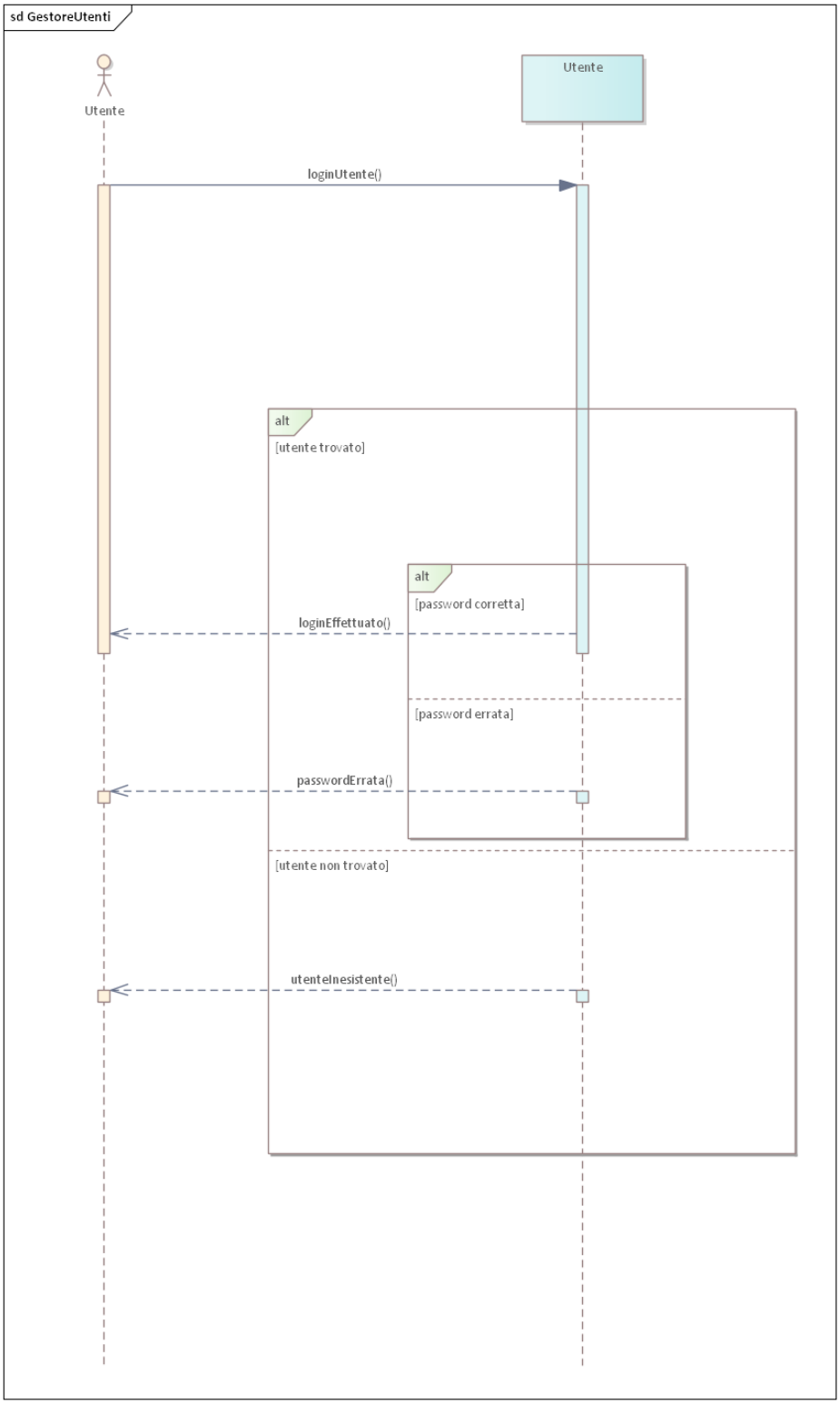


Figura 13: AccessoProfiloUtente

5.2.2 AggiungiProdotto

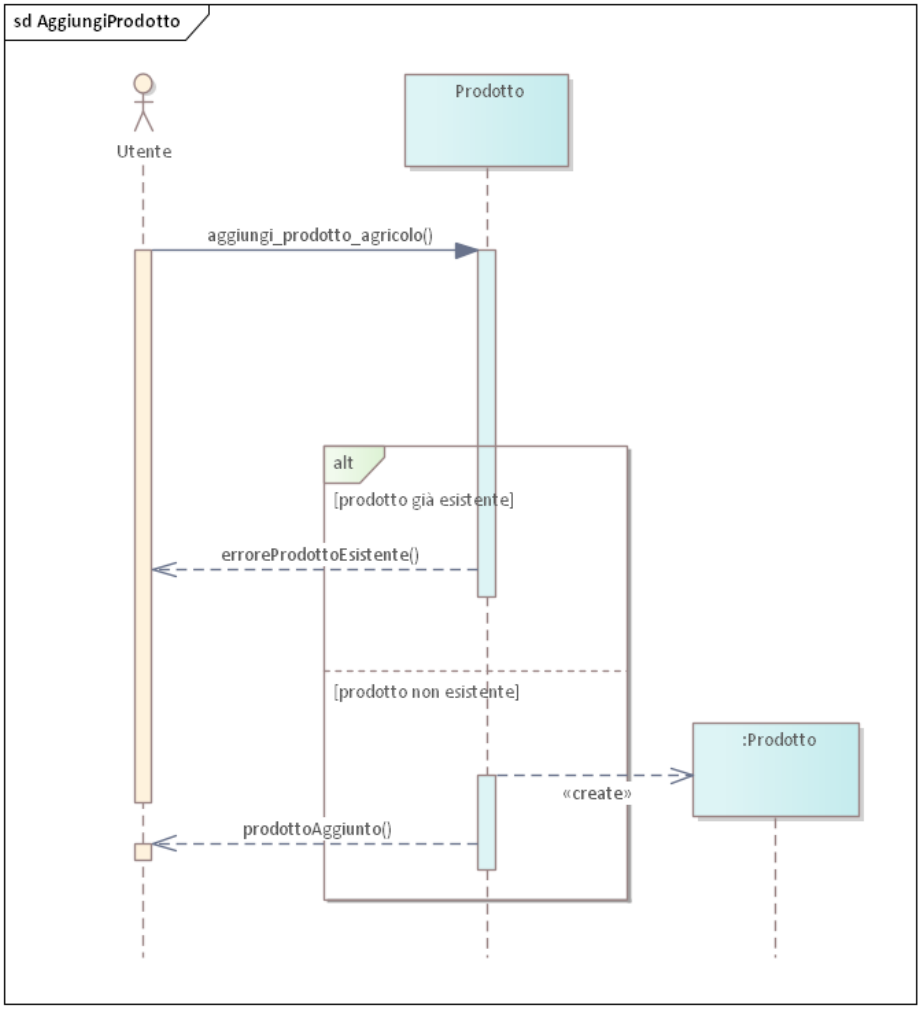


Figura 14: AggiungiProdotto

5.2.3 RegistraSpeseTasse

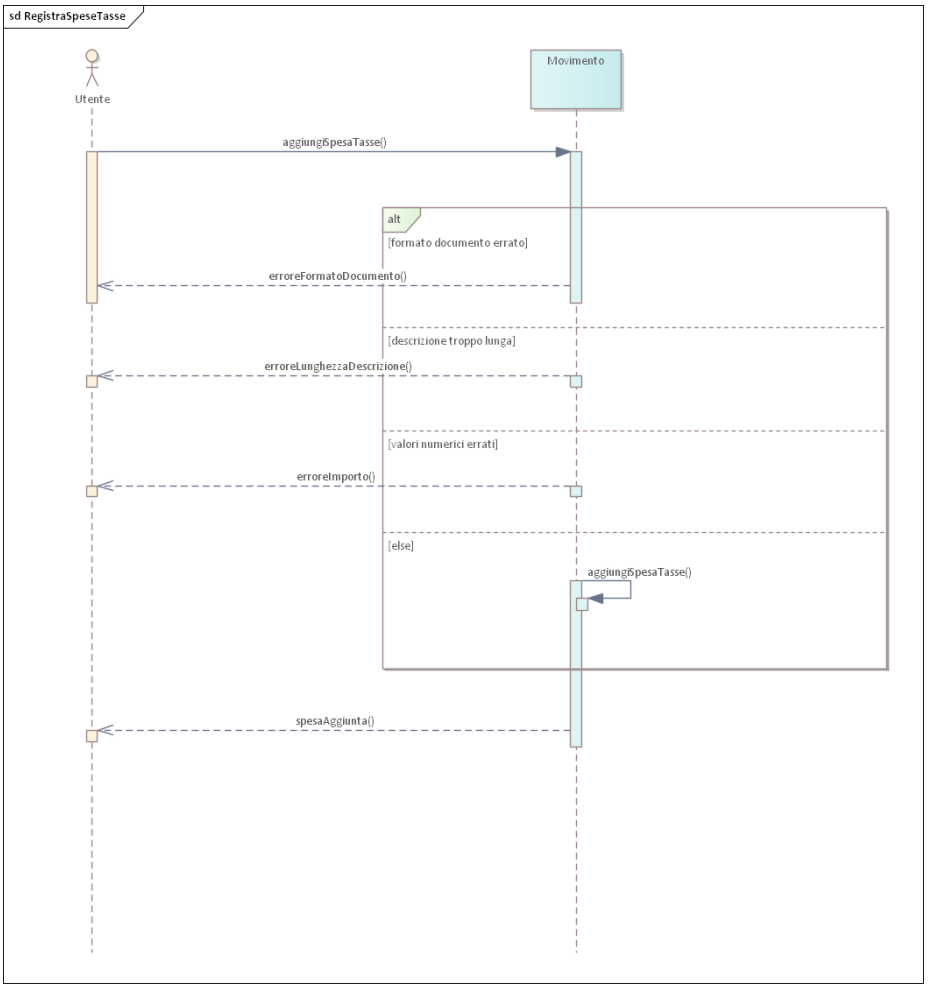


Figura 15: RegistraSpeseTasse

5.2.4 GuadagnoAziendale

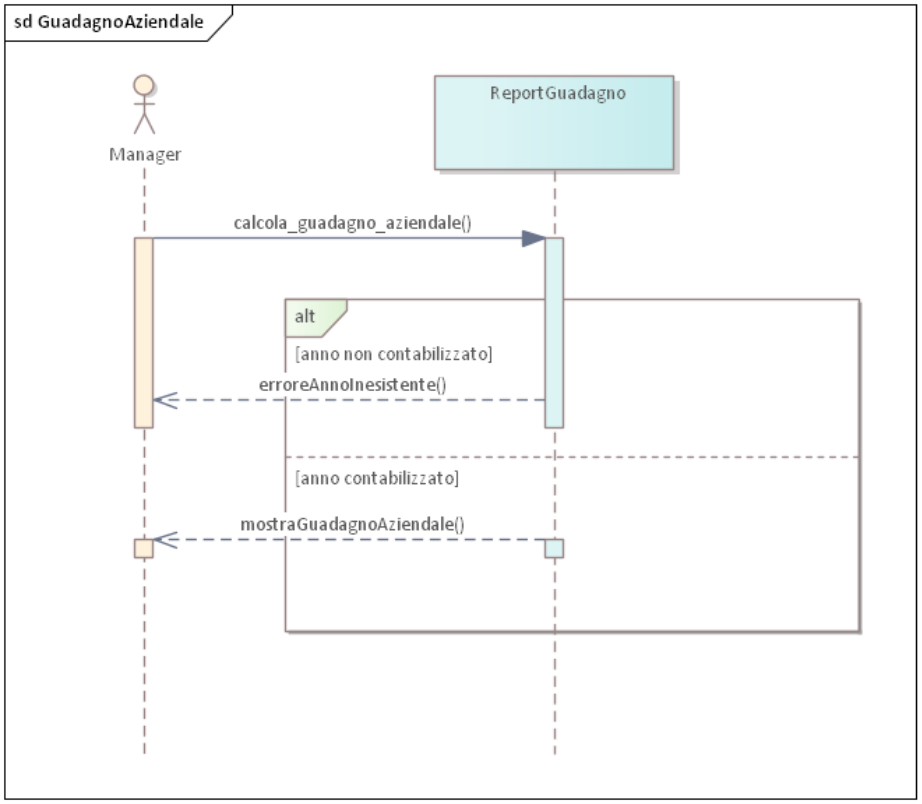


Figura 16: GuadagnoAziendale

5.3 Diagrammi di attività

5.3.1 Accessi Profilo Utente

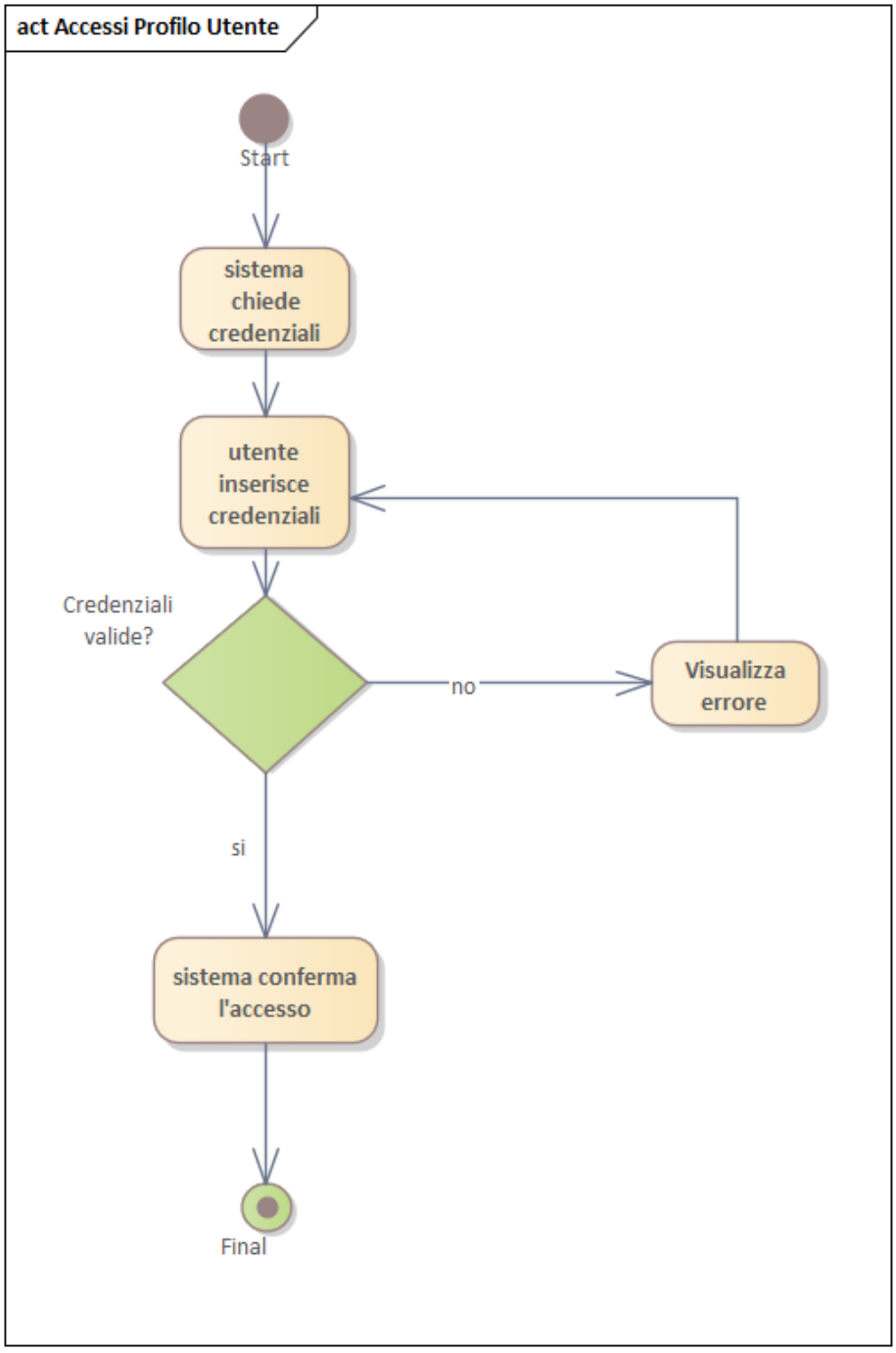


Figura 17: Accessi Profilo Utente

5.3.2 Aggiungi Prodotto

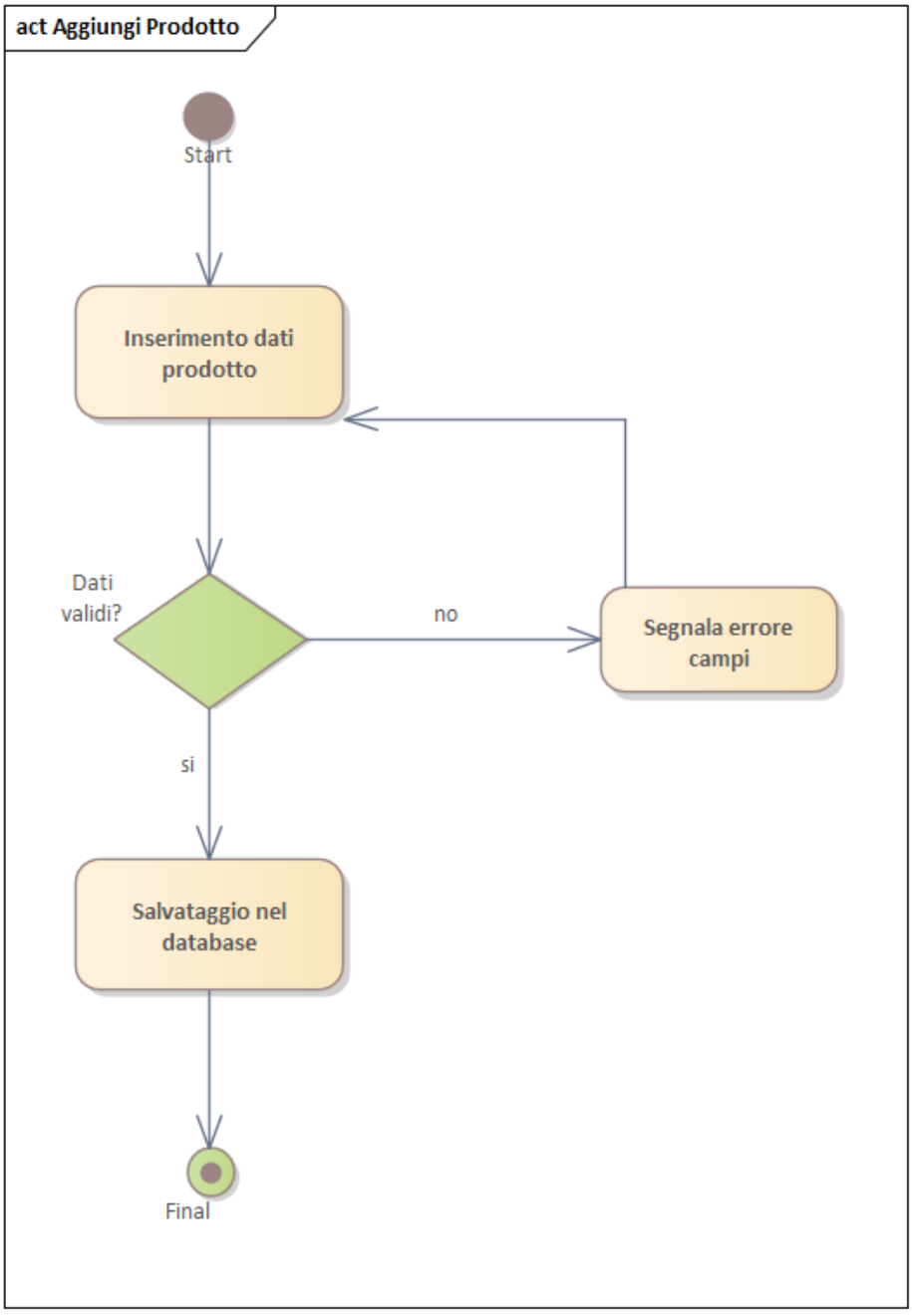


Figura 18: Aggiungi Prodotto

5.3.3 Guadagno Aziendale

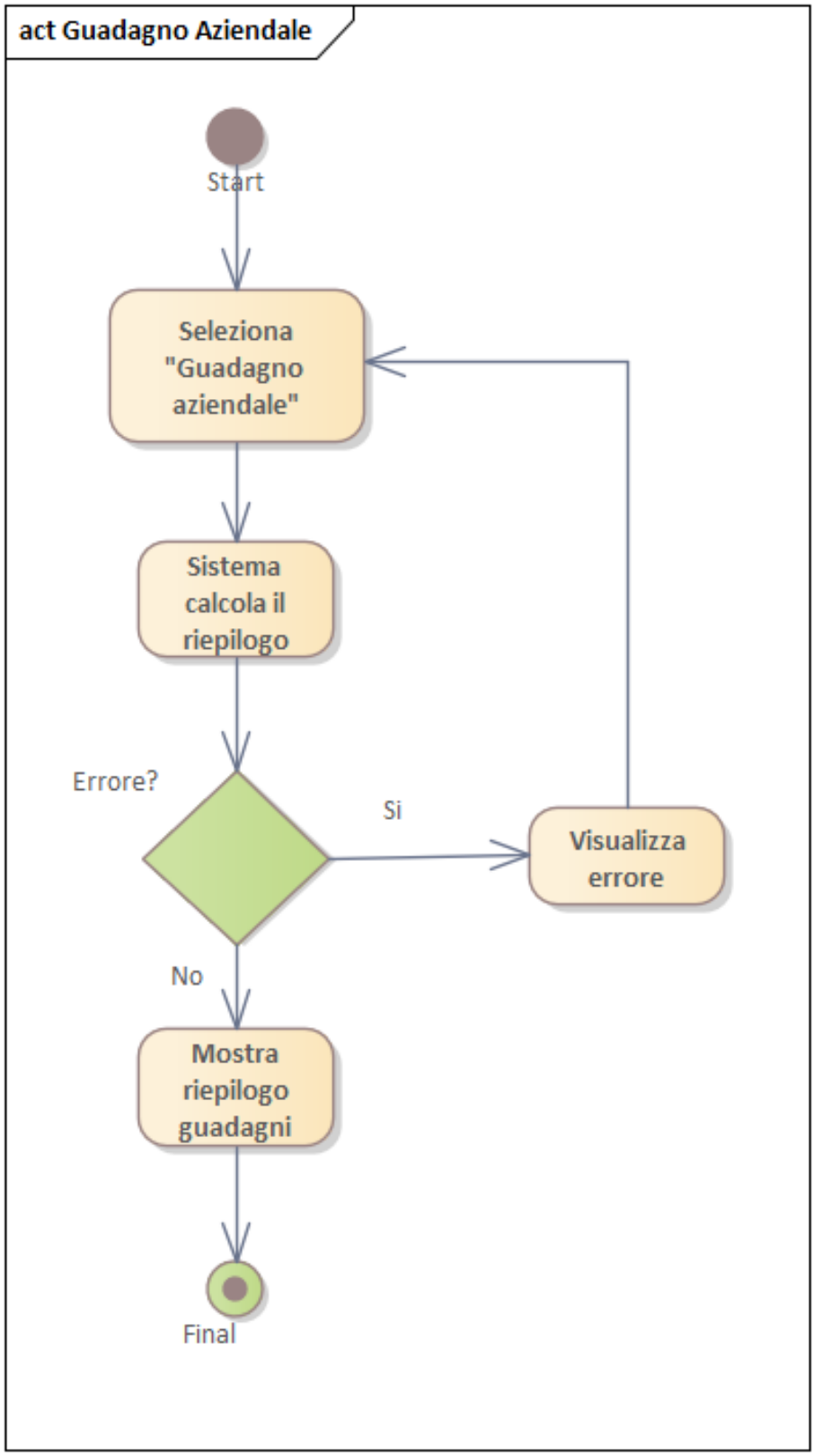


Figura 19: Guadagno Aziendale

5.3.4 Registra Entrate

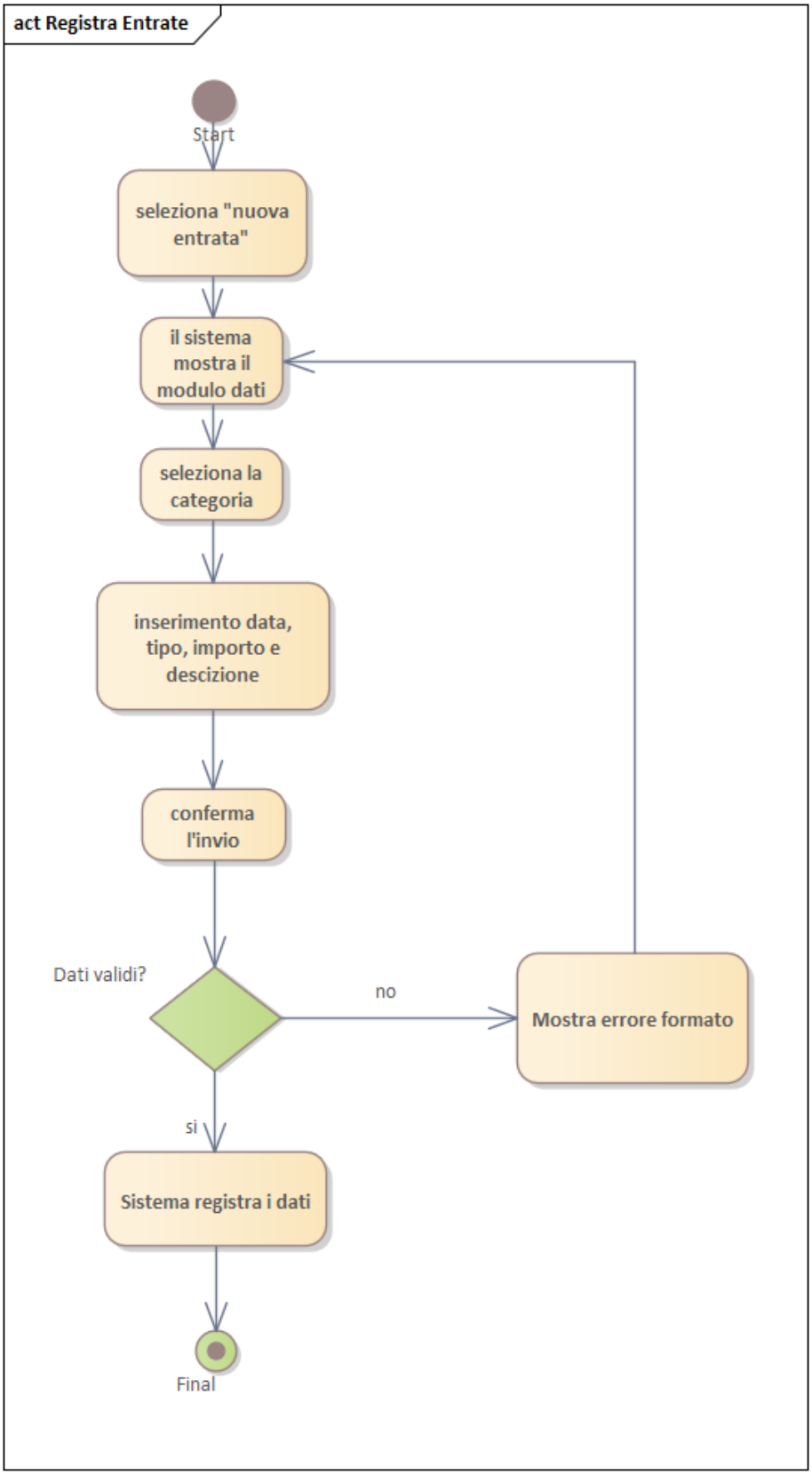


Figura 20: Registra Entrate

5.3.5 Registra Uscite

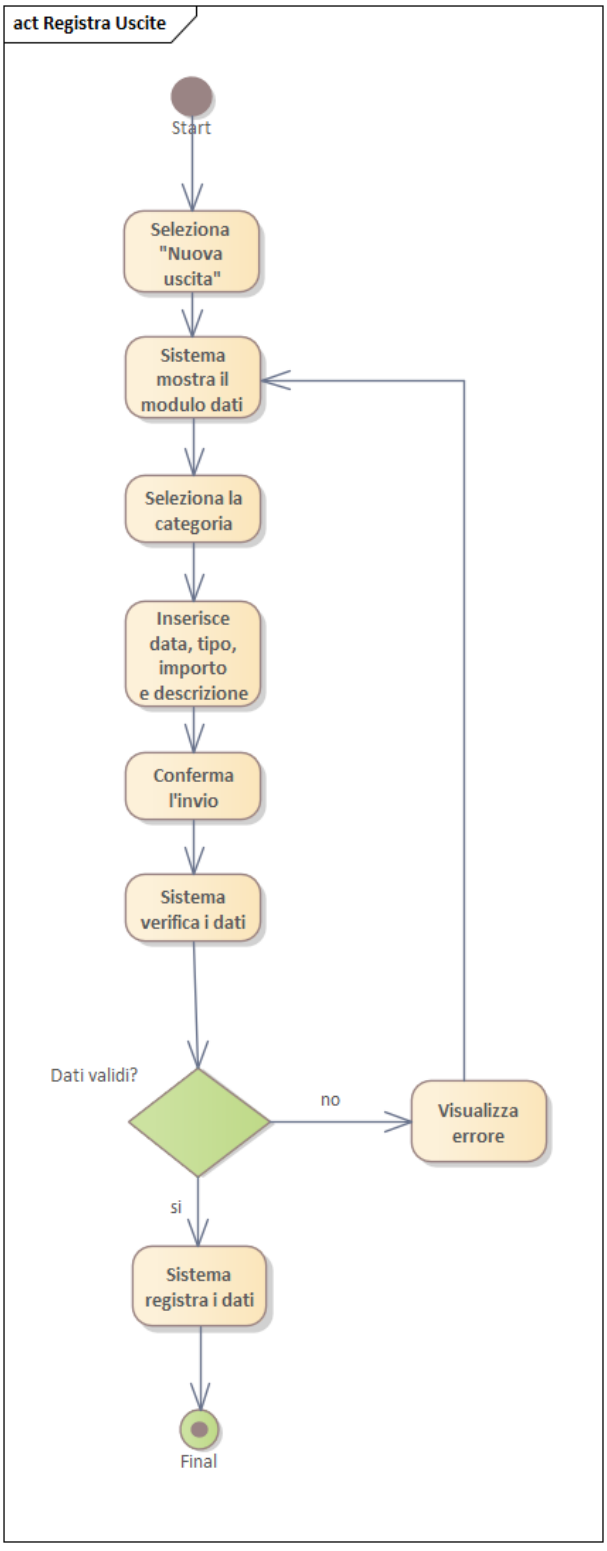


Figura 21: Registra Uscite

6 Diagrammi di progettazione

6.1 Diagramma delle classi di progettazione

6.1.1 Package di progettazione

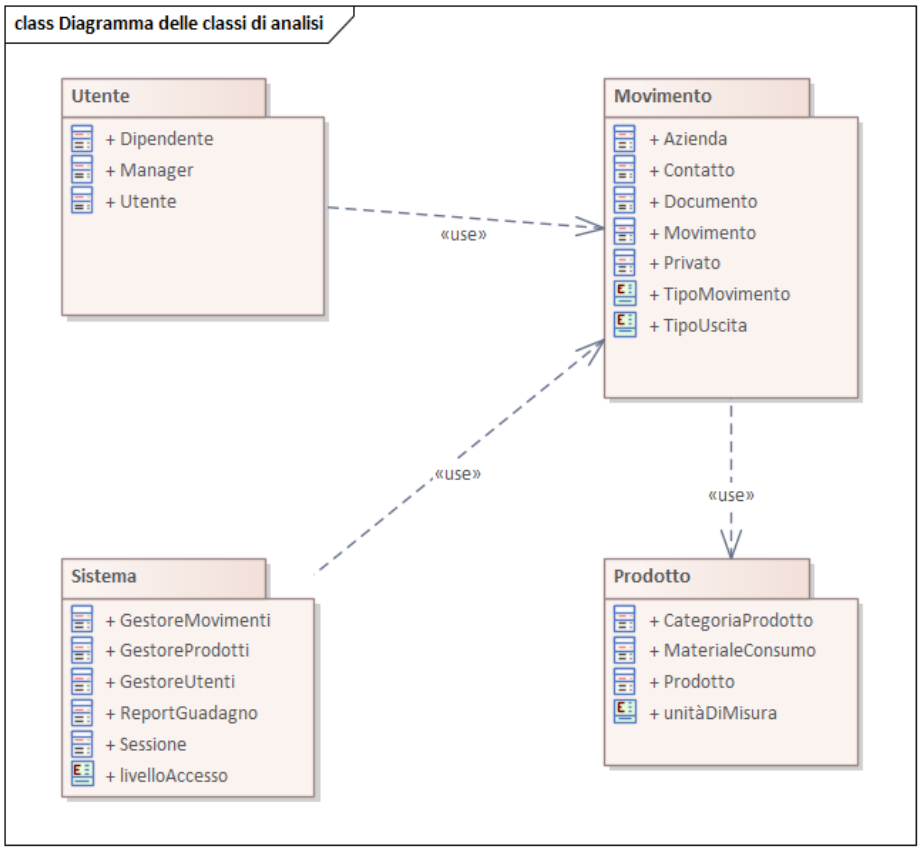


Figura 22: Diagramma delle classi di progettazione

6.1.2 Package di progettazione: Utente

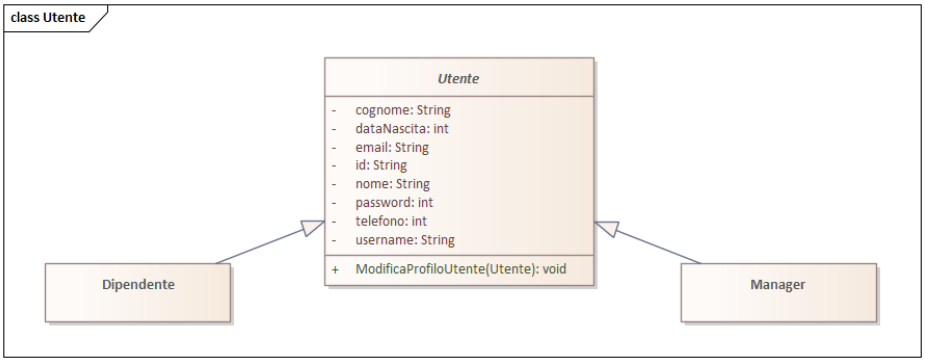


Figura 23: Diagramma delle classi di progettazione - Package Utente

6.1.3 Package di progettazione: Movimento

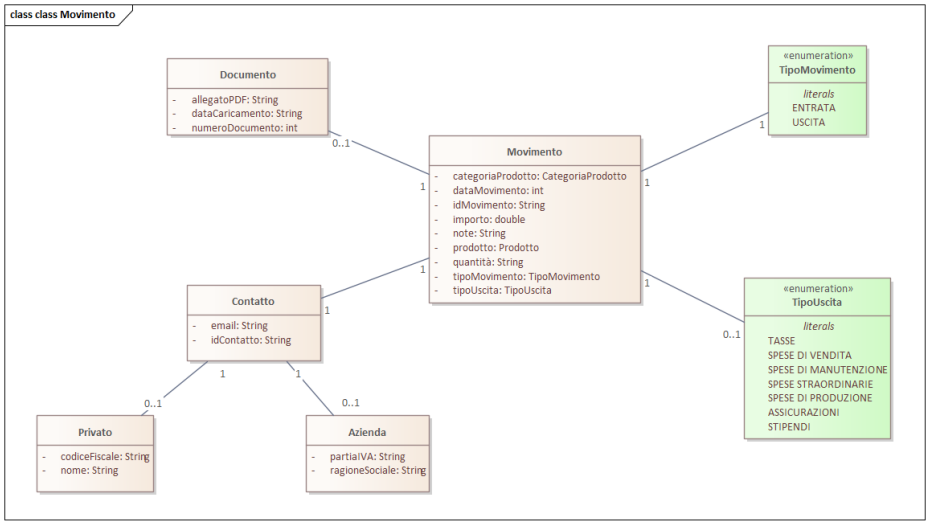


Figura 24: Diagramma delle classi di progettazione - Package Movimento

6.1.4 Package di progettazione: Prodotto

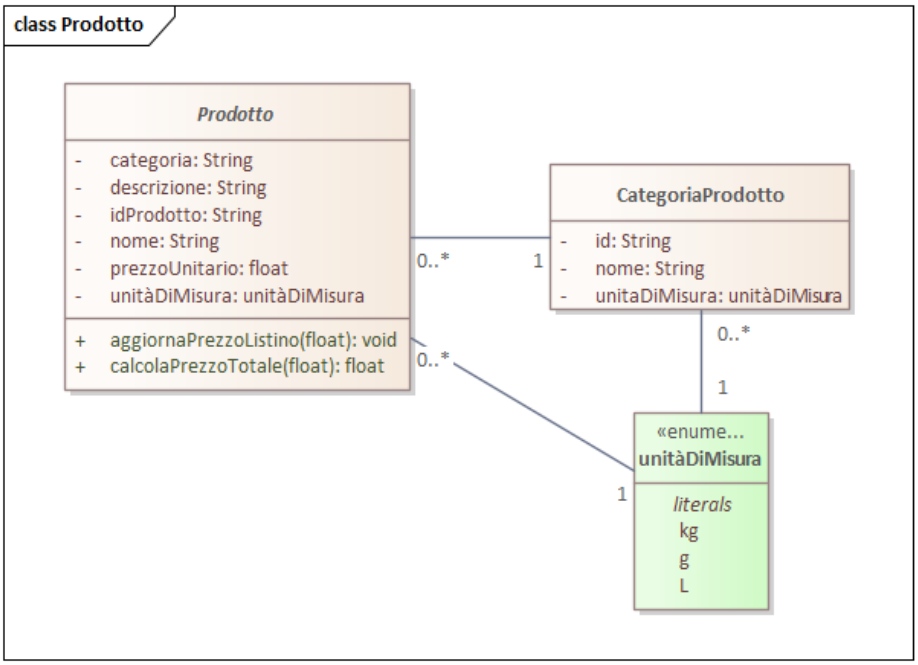


Figura 25: Diagramma delle classi di progettazione - Package Prodotto

6.1.5 Package di progettazione: Sistema

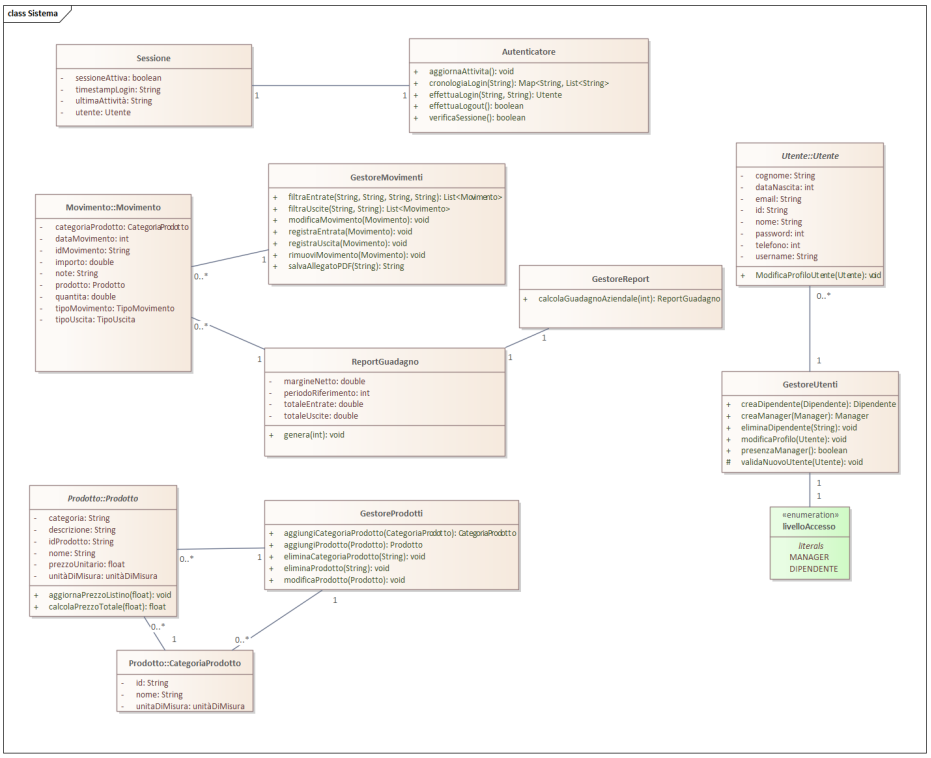
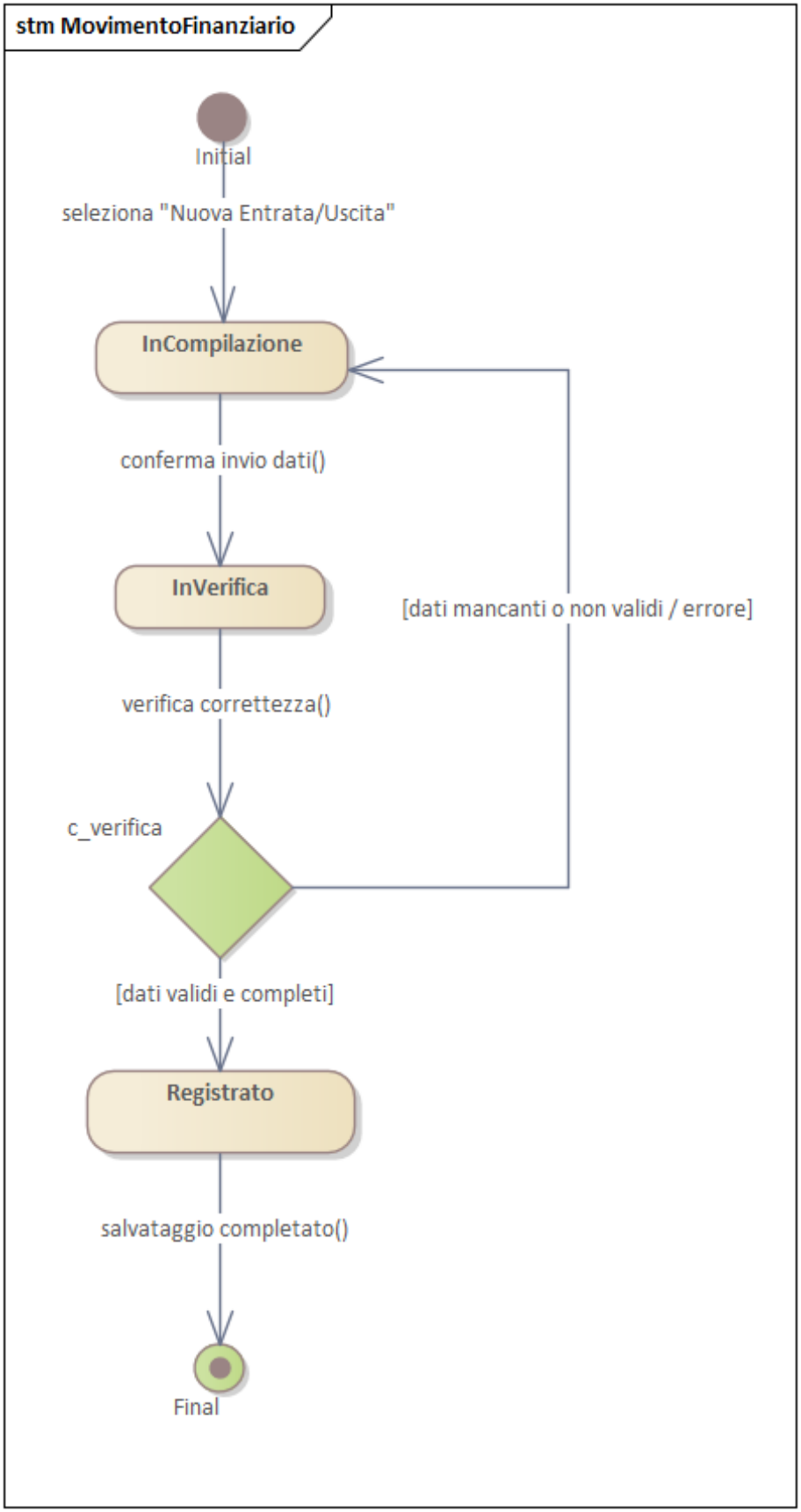


Figura 26: Diagramma delle classi di progettazione - Package Sistema

6.2 Diagrammi delle macchine a stati

6.2.1 Movimento Finanziario



6.2.2 Profilo Utente

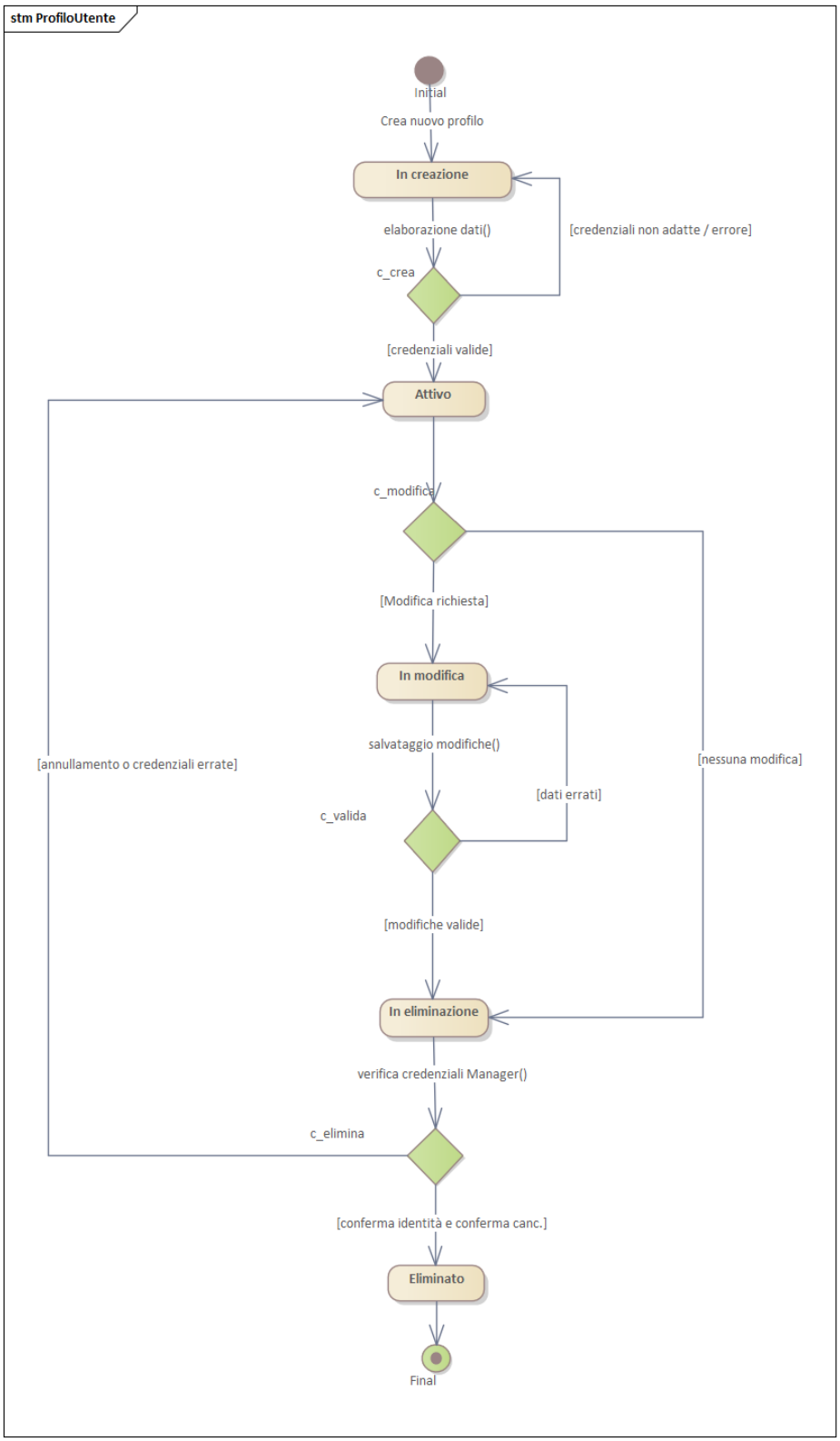


Figura 28: Profilo Utente

6.2.3 Sessione Utente

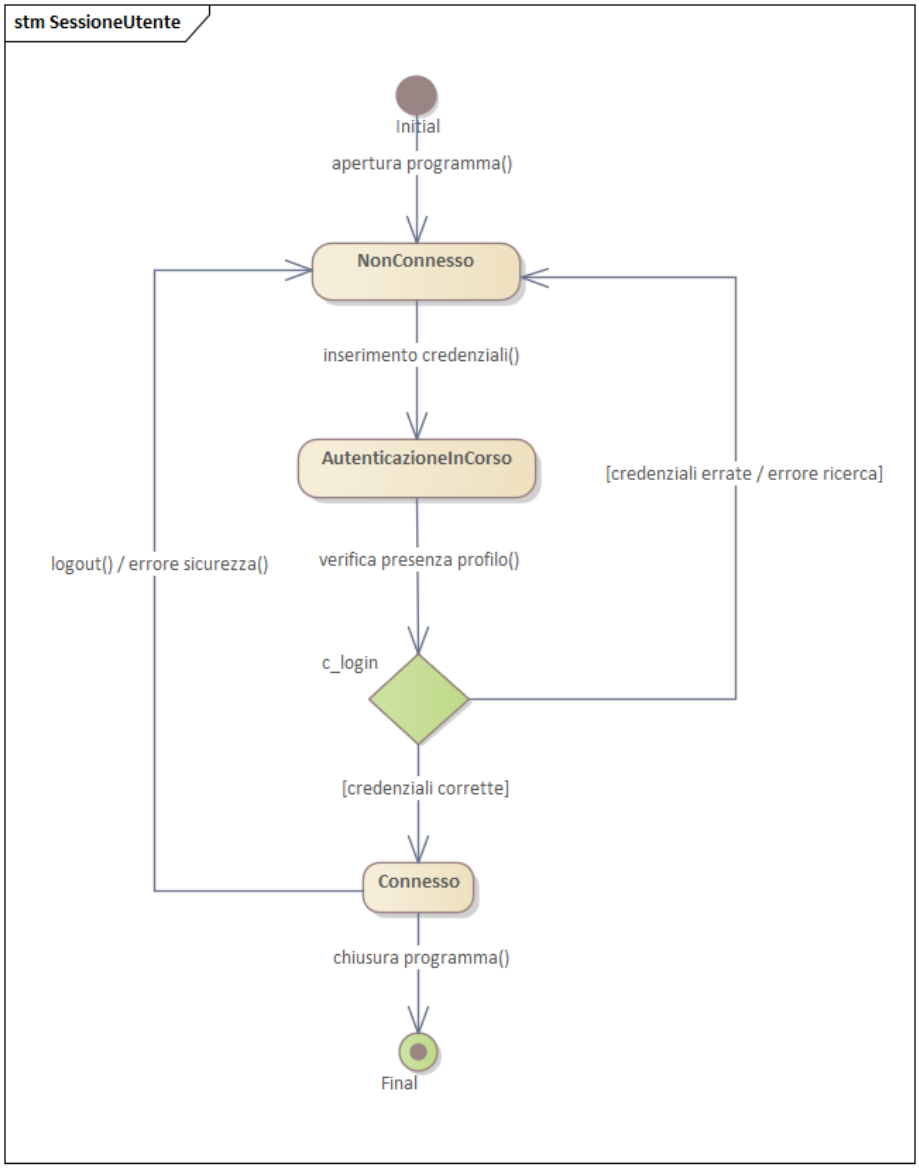


Figura 29: Sessione Utente

6.3 Diagramma dei componenti

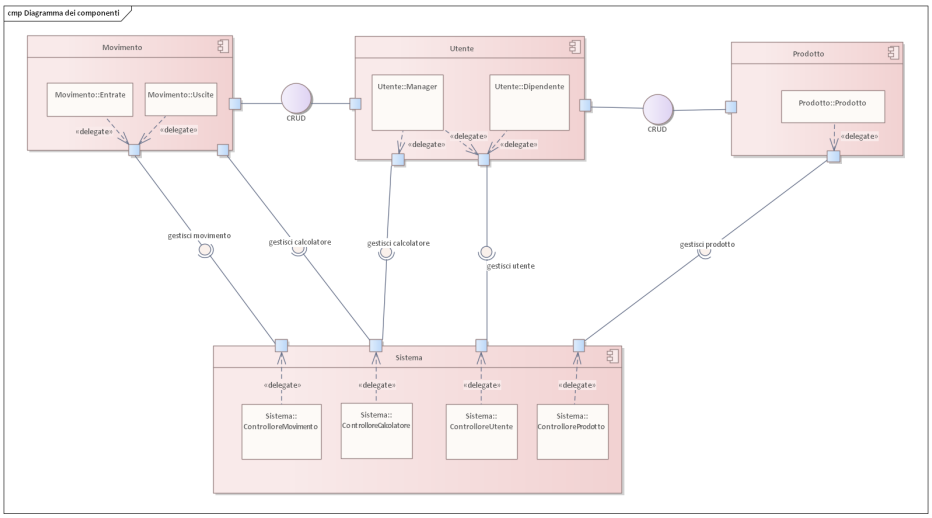


Figura 30: Diagramma dei componenti

6.4 Diagramma di entità-relazione

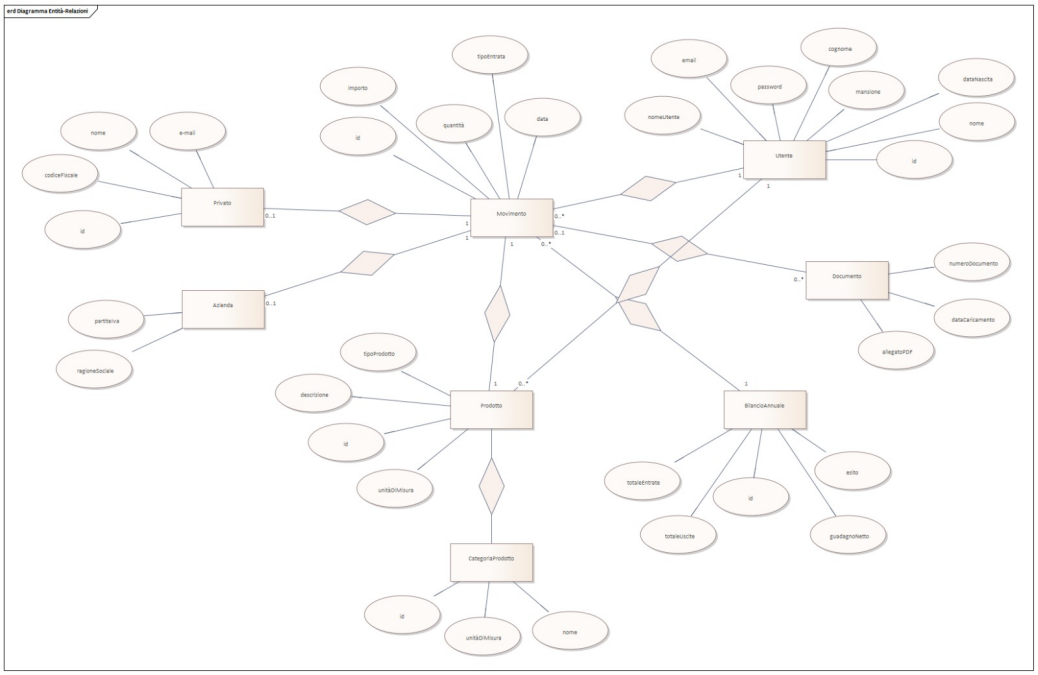


Figura 31: Diagramma di entità-relazione

7 Implementazione

7.1 Diagramma di deployment

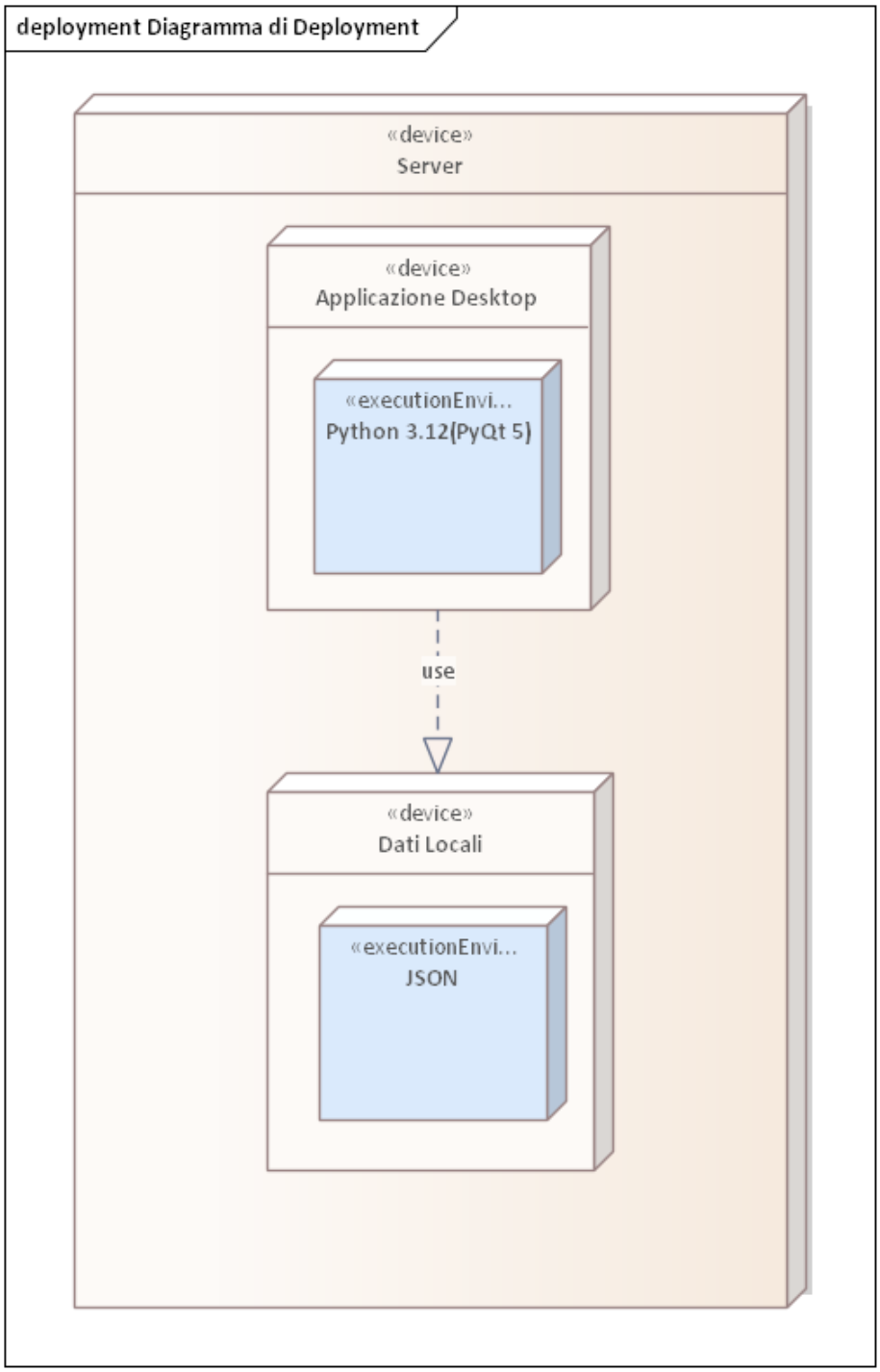


Figura 32: Diagramma di deployment

7.2 Mockup

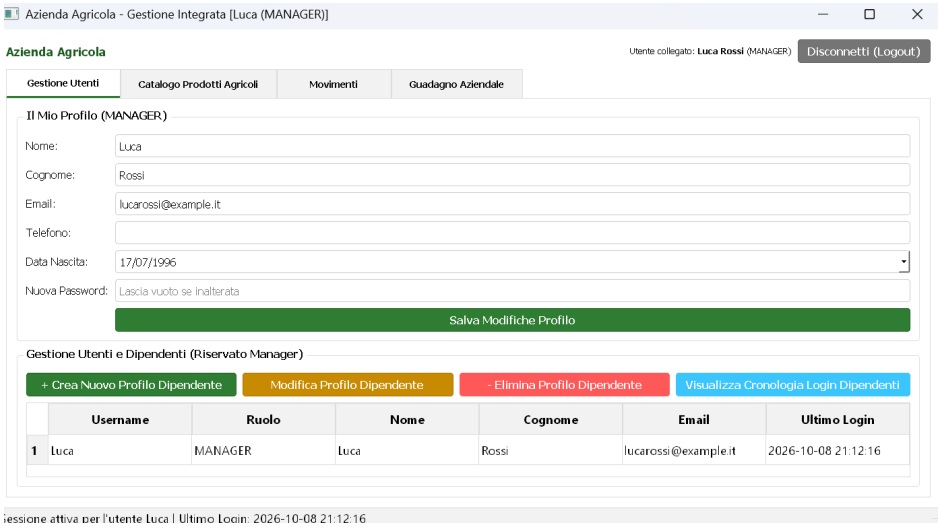


Figura 33: Gestione Utenti

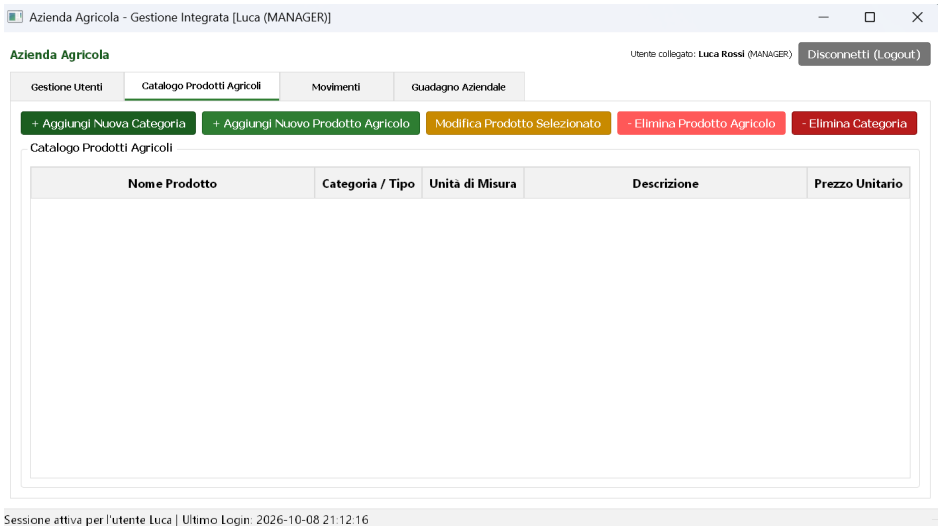


Figura 34: Catalogo Prodotti Agricoli

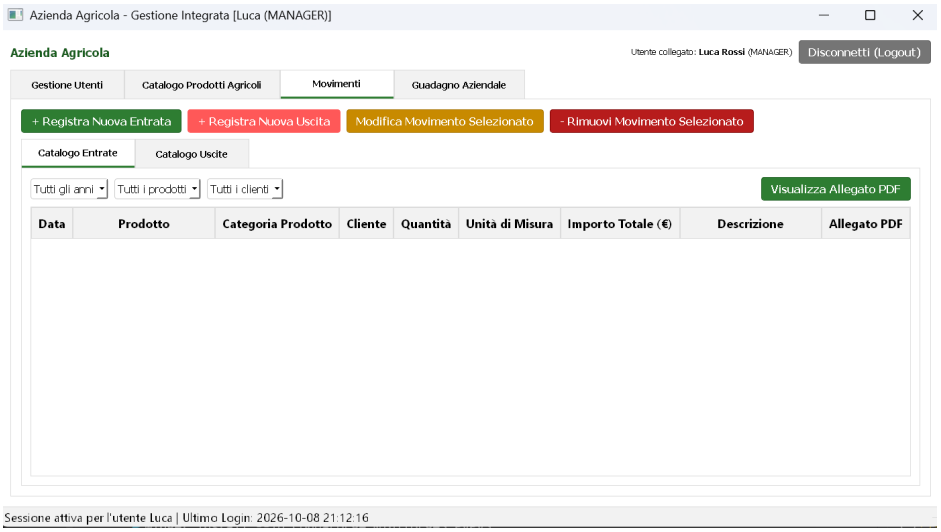


Figura 35: Movimenti

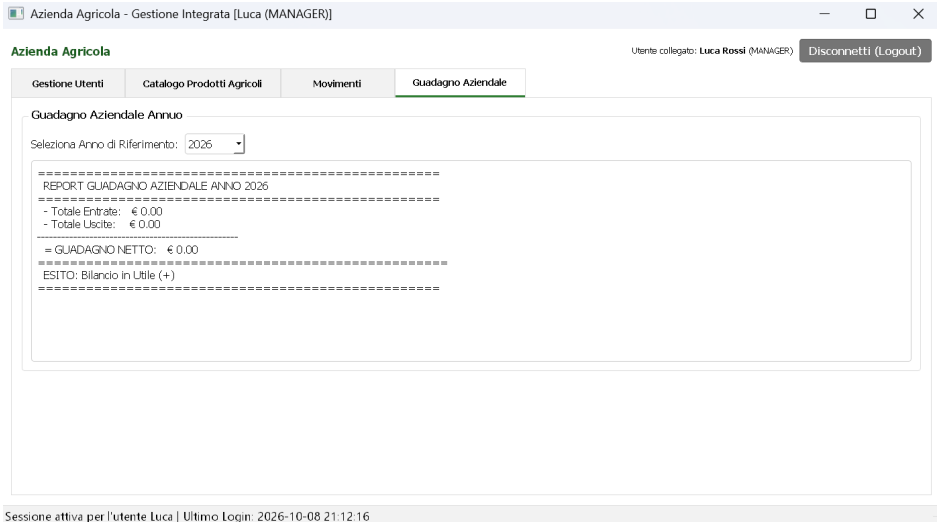


Figura 36: Guadagno aziendale

7.3 Descrizione unit tests

I seguenti test rappresentano un insieme di unit test scritti in Python utilizzando il framework unittest, con l'obiettivo di garantire il corretto funzionamento delle principali componenti dell'applicazione desktop per la gestione di un'azienda agricola. Sono state verificate le classi del dominio, i servizi applicativi, la persistenza dei dati nei file JSON e le principali interfacce grafiche PyQt5. In particolare, i test controllano la gestione degli utenti e dei relativi ruoli, l'autenticazione, la gestione di prodotti e categorie, la registrazione dei movimenti finanziari e la generazione dei report, contribuendo a individuare eventuali errori durante lo sviluppo.

7.3.1 Domain test

- Unit test puri sui modelli del dominio applicativo.
- Valida regole di business e requisiti specifici.

```
    class TestDomainModels(unittest.TestCase):
    def test_valida_password_rnf3(self):
        u = Utente("1", "user", "Pass1234", "N", "C", "e@a.it", "123", "
            1990-01-01", livelloAccesso.DIPENDENTE)

        u.modificaProfiloUtente("N", "C", "e@a.it", "123", "1990-01-01", "
            NewPass123")
        self.assertEqual(u.password, "NewPass123")
        u.modificaProfiloUtente("N", "C", "e@a.it", "123", "1990-01-01", "
            abcdefgh1")
        self.assertEqual(u.password, "abcdefgh1")

        with self.assertRaises(ValueError):
            u.modificaProfiloUtente("N", "C", "e@a.it", "123", "1990-01-01",
                "Pass1")

        with self.assertRaises(ValueError):
            u.modificaProfiloUtente("N", "C", "e@a.it", "123", "1990-01-01",
                "Password123!")
        with self.assertRaises(ValueError):
            u.modificaProfiloUtente("N", "C", "e@a.it", "123", "1990-01-01",
                "Pass 1234")

    def test_movimento_descrizione_limit_rnf8(self):
        desc_ok = "A" * 500
        m = Movimento(
            idMovimento="M1",
            tipo=TipoMovimento.ENTRATA,
            quantita=10.0,
            prezzoTotale=100.0,
            dataMovimento="2026-07-22",
            descrizione=desc_ok
        )
        self.assertEqual(len(m.descrizione), 500)

        desc_too_long = "A" * 501
```

```
        with self.assertRaises(ValueError):
            Movimento(
                idMovimento="M2",
                tipo=TipoMovimento.ENTRATA,
                quantita=10.0,
                prezzoTotale=100.0,
                dataMovimento="2026-07-22",
                descrizione=desc_too_long
            )

    def test_prodotto_calcoli(self):
        p = Prodotto(
            idProdotto="P1",
            nome="Olio EVO",
            descrizione="Olio Extravergine",
            prezzoUnitario=12.0,
            categoria="Olio",
            unitaMisura="litri"
        )
        self.assertEqual(p.calcolaPrezzoTotale(5), 60.0)

    def test_report_guadagno_rf29(self):
        movs = [
            Movimento("M1", TipoMovimento.ENTRATA, 10, 500.0, "2026-05-10",
                "Vendita olio"),
            Movimento("M2", TipoMovimento.ENTRATA, 5, 200.0, "2026-06-15", "
                Vendita vino"),
            Movimento("M3", TipoMovimento.USCITA, 1, 150.0, "2026-04-01", "
                Acquisto concime"),
            Movimento("M4", TipoMovimento.USCITA, 1, 50.0, "2025-04-01", "
                Spesa anno precedente")
        ]

        rep2026 = ReportGuadagno.genera(2026, movs)
        self.assertEqual(rep2026.totaleEntrate, 700.0)
        self.assertEqual(rep2026.totaleUscite, 150.0)
        self.assertEqual(rep2026.margineNetto, 550.0)
```

7.3.2 Services tests

- Unit test della logica applicativa.
- Verifica i servizi di autenticazione.

```
    class TestServices(unittest.TestCase):
    def setUp(self):
        self.temp_dir = tempfile.mkdtemp()
        self.repo = DataRepository(data_dir=self.temp_dir)
        self.auth_service = Autenticatore(self.repo)
        self.user_manager = GestoreUtenti(self.repo)
        self.product_service = GestoreProdotti(self.repo)
        self.financial_service = GestoreMovimenti(self.repo)

    def tearDown(self):
        shutil.rmtree(self.temp_dir)

    def test_registrazione_primo_manager_e_dipendente(self):
        self.assertFalse(self.user_manager.presenzaManager())
```

```
        manager = self.user_manager.creaManager(
            username="manager1",
            password="Password123",
            nome="Mario",
            cognome="Rossi",
            email="mario.rossi@azienda.it",
            telefono="3331234567",
            dataNascita="1980-01-01"
        )
        self.assertEqual(manager.ruolo, livelloAccesso.MANAGER)
        self.assertTrue(self.user_manager.presenzaManager())

        with self.assertRaises(ValueError):
            self.user_manager.creaManager(
                "manager2", "Password123", "Luigi", "Verdi", "luigi@azienda.
                    it", "123", "1990-01-01"
            )

        dip = self.user_manager.creaDipendente(
            username="dipendente1",
            password="SecretPass1",
            nome="Anna",
            cognome="Bianchi",
            email="anna.bianchi@azienda.it",
            telefono="3409876543",
            dataNascita="1995-05-05"
        )
        self.assertEqual(dip.ruolo, livelloAccesso.DIPENDENTE)

    def test_unicita_email_rnf4(self):
        self.user_manager.creaManager(
            "m1", "Pass1234", "M", "R", "mario@azienda.it", "123", "
                1980-01-01"
        )

        with self.assertRaises(ValueError):
            self.user_manager.creaDipendente(
                "m2", "Pass1234", "A", "B", "mario@azienda.it", "456", "
                    1990-01-01"
            )

    def test_data_nascita_obbligatoria(self):
        with self.assertRaises(ValueError):
            self.user_manager.creaManager(
                "m_empty", "Password123", "Mario", "Rossi", "
                    empty_dob@azienda.it", "123", ""
            )

        self.user_manager.creaManager(
            "m1", "Password123", "Mario", "Rossi", "mario@azienda.it", "123"
                , "01/01/1980"
        )

        with self.assertRaises(ValueError):
            self.user_manager.creaDipendente(
                "d1", "SecretPass1", "Anna", "Bianchi", "anna@azienda.it", "
                    456", "   "
            )

    def test_impossibile_aggiungere_prodotto_senza_categoria(self):
        with self.assertRaises(ValueError) as ctx:
```

```
            self.product_service.aggiungiProdotto(
                nome="Mele Golden", descrizione="Frutta fresca", prezzo=1.5,
                    unita="kilogrammi", tipo="FRUTTA"
            )
        self.assertIn("categoria", str(ctx.exception).lower())

        self.product_service.aggiungiCategoria("FRUTTA", "kilogrammi")
        with self.assertRaises(ValueError) as ctx2:
            self.product_service.aggiungiProdotto(
                nome="Mele Golden", descrizione="Frutta fresca", prezzo=1.5,
                    unita="litri", tipo="NON_ESISTE"
            )
        self.assertIn("non esiste", str(ctx2.exception).lower())

    def test_unicita_prodotto_rnf5(self):
        self.product_service.aggiungiCategoria("VINO", "litri")
        with self.assertRaises(ValueError):
            self.product_service.aggiungiCategoria("VINO", "litri")

        p1 = self.product_service.aggiungiProdotto(
            nome="Vino Chianti", descrizione="Rosso DOCG", prezzo=15.0,
                unita="litri", tipo="VINO"
        )
        self.assertIsNotNone(p1)

        with self.assertRaises(ValueError):
            self.product_service.aggiungiProdotto(
                nome="Vino Chianti", descrizione="Altro", prezzo=20.0, unita
                    ="litri", tipo="VINO"
            )

    def test_login_logout(self):
        self.user_manager.creaManager(
            "manager1", "Password123", "Mario", "Rossi", "mario@azienda.it",
                 "123", "1980-01-01"
        )

        user = self.auth_service.effettuaLogin("manager1", "Password123")
        self.assertIsNotNone(user)
        self.assertTrue(self.auth_service.verificaSessione())

        self.assertTrue(self.auth_service.effettuaLogout())
        self.assertFalse(self.auth_service.verificaSessione())

    def test_sessione_timeout_inattivita(self):
        self.user_manager.creaManager(
            "m_timeout", "Password123", "Mario", "Rossi", "mario@azienda.it"
                , "123", "1980-01-01"
        )
        self.auth_service.effettuaLogin("m_timeout", "Password123")
        self.assertEqual(self.auth_service.session_timeout_minutes, 10)
        self.assertTrue(self.auth_service.verificaSessione())

        past_dt = datetime.datetime.now() - datetime.timedelta(minutes=11)
        self.auth_service._last_activity_dt = past_dt
        self.auth_service.current_session.ultimaAttivita = past_dt.strftime(
            "%Y-%m-%d %H:%M:%S")

        self.assertFalse(self.auth_service.verificaSessione())

    def test_registrazione_entrate_e_uscite(self):
        cat = self.product_service.aggiungiCategoria("OLIO", "litri")
```

```
        prod = self.product_service.aggiungiProdotto(
            nome="Olio di Oliva",
            descrizione="Extravergine",
            prezzo=12.0,
            unita="litri",
            tipo="OLIO"
        )

        e = self.financial_service.registraEntrata(
            categoria_prodotto="OLIO",
            prodotto_id=prod.idProdotto,
            cliente_tipo="Azienda",
            importo=1200.0,
            data="2026-07-20",
            descrizione="Fornitura olio extravergine ristorante",
            cliente_dettagli={"ragioneSociale": "Ristorante Da Mario", "
                partitaIVA": "12345678901"}
        )
        self.assertEqual(e.prezzoTotale, 1200.0)
        self.assertEqual(e.prodottoId, prod.idProdotto)
        self.assertEqual(e.prodottoNome, "Olio di Oliva")

        u = self.financial_service.registraUscita(
            categoria_uscita="SPESE DI PRODUZIONE",
            prodotto_id=prod.idProdotto,
            importo=350.0,
            data="2026-07-21",
            descrizione="Acquisto bottiglie olio",
            fornitore_note="AgriBio Srl"
        )
        self.assertEqual(u.prezzoTotale, 350.0)
        self.assertEqual(u.prodottoId, prod.idProdotto)

        movs = self.financial_service.getAllMovements()
        self.assertEqual(len(movs), 2)

    def test_modifica_movimento(self):
        e = self.financial_service.registraEntrata(
            categoria_prodotto="FRUTTA",
            prodotto_id=None,
            cliente_tipo="Privato",
            importo=50.0,
            data="2026-08-01",
            descrizione="Vendita mele",
            quantita=10.0
        )

        mod = self.financial_service.modificaMovimento(
            movimento_id=e.idMovimento,
            data="2026-08-02",
            importo=75.0,
            quantita=15.0,
            descrizione="Vendita mele e pere"
        )
        self.assertEqual(mod.dataMovimento, "2026-08-02")
        self.assertEqual(mod.prezzoTotale, 75.0)
        self.assertEqual(mod.quantita, 15.0)
        self.assertEqual(mod.descrizione, "Vendita mele e pere")

        loaded = self.financial_service.getAllMovements()
        target = next((m for m in loaded if m.idMovimento == e.idMovimento),
             None)
```

```
        self.assertIsNotNone(target)
        self.assertEqual(target.prezzoTotale, 75.0)

        with self.assertRaises(ValueError):
            self.financial_service.modificaMovimento(e.idMovimento, "
                2026-08-02", -10.0, 5.0, "Err")

        with self.assertRaises(ValueError):
            self.financial_service.modificaMovimento(e.idMovimento, "
                2026-08-02", 10.0, -5.0, "Err")

        with self.assertRaises(ValueError):
            self.financial_service.modificaMovimento(e.idMovimento, "
                2026-08-02", 10.0, 5.0, "A" * 501)

        with self.assertRaises(ValueError):
            self.financial_service.modificaMovimento("ID-NON-ESISTE", "
                2026-08-02", 10.0, 5.0, "Desc")

    def test_categorie_dinamiche_e_unita_misura(self):
        cat = self.product_service.aggiungiCategoria("MIELE", "grammi")
        self.assertEqual(cat.nome, "MIELE")
        self.assertEqual(cat.unitaMisura, "grammi")

        with self.assertRaises(ValueError):
            self.product_service.aggiungiCategoria("MIELE", "litri")

        with self.assertRaises(ValueError):
            self.product_service.aggiungiCategoria("VINO", "bottiglie")

        cats = self.product_service.getAllCategories()
        miele_cat = next((c for c in cats if c.nome == "MIELE"), None)
        self.assertIsNotNone(miele_cat)

        prod = self.product_service.aggiungiProdotto(
            nome="Miele Millefiori",
            descrizione="Vasetto 500g",
            prezzo=6.50,
            unita=miele_cat.unitaMisura,
            tipo="MIELE"
        )
        self.assertEqual(prod.unitaMisura, "grammi")
        self.assertEqual(prod.categoria, "MIELE")

    def test_singolo_manager_vincolo(self):
        m1 = self.user_manager.creaManager(
            "boss", "Password123", "Boss", "Unico", "boss@azienda.it", "123"
                , "1985-01-01"
        )
        self.assertIsNotNone(m1.id)
        self.assertEqual(len(m1.id), 8)

        with self.assertRaises(ValueError):
            self.user_manager.creaManager(
                "boss2", "Password123", "Boss2", "Due", "boss2@azienda.it",
                    "123", "1986-01-01"
            )

        with self.assertRaises(ValueError) as ctx:
            self.user_manager.creaManager(
                "boss3", "Password123", "Boss3", "Tre", "boss3@azienda.it",
                    "123", "1987-01-01"
```

```
            )
        self.assertIn("manager", str(ctx.exception).lower())

    def test_prodotto_unificato_backend(self):
        from app.models import Prodotto
        self.product_service.aggiungiCategoria("ORTAGGI", "kilogrammi")
        p = self.product_service.aggiungiProdotto(
            nome="Carote",
            descrizione="Carote fresche",
            prezzo=1.80,
            unita="kilogrammi",
            tipo="ORTAGGI"
        )
        self.assertIsInstance(p, Prodotto)
        self.assertEqual(p.categoria, "ORTAGGI")
        self.assertEqual(p.unitaMisura, "kilogrammi")

        loaded = self.repo.caricaProdotti()
        self.assertEqual(len(loaded), 1)
        self.assertIsInstance(loaded[0], Prodotto)
        self.assertEqual(loaded[0].idProdotto, p.idProdotto)
        self.assertEqual(loaded[0].categoria, "ORTAGGI")

    def test_unicita_nome_prodotto(self):
        self.product_service.aggiungiCategoria("MIELE", "kilogrammi")
        p1 = self.product_service.aggiungiProdotto(
            nome="Miele Millefiori", descrizione="Vasetto", prezzo=5.0,
                unita="kilogrammi", tipo="MIELE"
        )
        self.assertEqual(p1.nome, "Miele Millefiori")

        with self.assertRaises(ValueError) as ctx:
            self.product_service.aggiungiProdotto(
                nome="Miele Millefiori", descrizione="Altro vasetto", prezzo
                    =6.0, unita="kilogrammi", tipo="MIELE"
            )
        self.assertIn("esiste gi  ", str(ctx.exception).lower())

    def test_id_univoci_dopo_cancellazione(self):
        m = self.user_manager.creaManager(
            "m1", "Pass1234", "M", "R", "m1@azienda.it", "1", "1980-01-01"
        )
        d1 = self.user_manager.creaDipendente(
            "d1", "Pass1234", "D1", "B", "d1@azienda.it", "2", "1990-01-01"
        )
        d2 = self.user_manager.creaDipendente(
            "d2", "Pass1234", "D2", "B", "d2@azienda.it", "3", "1991-01-01"
        )

        self.user_manager.eliminaDipendente(d2.id)
        users = self.user_manager.getAllUsers()
        self.assertEqual(len(users), 2)
        d3 = self.user_manager.creaDipendente(
            "d3", "Pass1234", "D3", "B", "d3@azienda.it", "4", "1992-01-01"
        )
        self.assertNotEqual(d3.id, d2.id)
        self.assertNotEqual(d3.id, d1.id)

        self.product_service.aggiungiCategoria("FRUTTA", "kilogrammi")
        prod1 = self.product_service.aggiungiProdotto("Mele", "Desc", 2.0, "
            kilogrammi", "FRUTTA")
        prod2 = self.product_service.aggiungiProdotto("Pere", "Desc", 2.5, "
```

```
            kilogrammi", "FRUTTA")

        self.product_service.eliminaProdotto(prod2.idProdotto)
        prod3 = self.product_service.aggiungiProdotto("Banane", "Desc", 3.0,
             "kilogrammi", "FRUTTA")
        self.assertNotEqual(prod3.idProdotto, prod2.idProdotto)
        self.assertNotEqual(prod3.idProdotto, prod1.idProdotto)

        mov1 = self.financial_service.registraEntrata(
            categoria_prodotto="FRUTTA",
            prodotto_id=prod1.idProdotto,
            cliente_tipo="Privato",
            importo=10.0,
            data="2026-09-11",
            descrizione="Vendita 1"
        )
        mov2 = self.financial_service.registraEntrata(
            categoria_prodotto="FRUTTA",
            prodotto_id=prod1.idProdotto,
            cliente_tipo="Privato",
            importo=20.0,
            data="2026-09-11",
            descrizione="Vendita 2"
        )

        self.financial_service.eliminaMovimento(mov2.idMovimento)
        mov3 = self.financial_service.registraUscita(
            categoria_uscita="SPESE DI MANUTENZIONE",
            prodotto_id=None,
            importo=5.0,
            data="2026-09-11",
            descrizione="Spesa"
        )
        self.assertNotEqual(mov3.idMovimento, mov2.idMovimento)
        self.assertNotEqual(mov3.idMovimento, mov1.idMovimento)

    def test_filtra_uscite(self):
        self.financial_service.registraUscita(
            categoria_uscita=TipoUscita.SPESE_DI_MANUTENZIONE.value,
            prodotto_id=None,
            importo=150.0,
            data="2026-03-10",
            descrizione="Manutenzione impianto"
        )
        self.financial_service.registraUscita(
            categoria_uscita=TipoUscita.SPESE_DI_PRODUZIONE.value,
            prodotto_id=None,
            importo=300.0,
            data="2026-04-15",
            descrizione="Concime"
        )
        self.financial_service.registraUscita(
            categoria_uscita=TipoUscita.SPESE_DI_MANUTENZIONE.value,
            prodotto_id=None,
            importo=80.0,
            data="2025-11-20",
            descrizione="Riparazione recinzione"
        )

        tutte = self.financial_service.filtraUscite()
        self.assertEqual(len(tutte), 3)
```

```
        uscite_2026 = self.financial_service.filtraUscite(anno="2026")
        self.assertEqual(len(uscite_2026), 2)

        manutenzione = self.financial_service.filtraUscite(tipo_spesa=
            TipoUscita.SPESE_DI_MANUTENZIONE.value)
        self.assertEqual(len(manutenzione), 2)

        comb = self.financial_service.filtraUscite(anno="2026", tipo_spesa=
            TipoUscita.SPESE_DI_MANUTENZIONE.value)
        self.assertEqual(len(comb), 1)
        self.assertEqual(comb[0].prezzoTotale, 150.0)

    def test_filtra_entrate(self):
        cat = self.product_service.aggiungiCategoria("VERDURA", "kg")
        prod = self.product_service.aggiungiProdotto(
            nome="Pomodori",
            descrizione="San Marzano",
            prezzo=2.5,
            unita="kg",
            tipo="VERDURA"
        )

        self.financial_service.registraEntrata(
            categoria_prodotto="VERDURA",
            prodotto_id=prod.idProdotto,
            cliente_tipo="Azienda",
            importo=500.0,
            data="2026-06-01",
            descrizione="Vendita ingrosso",
            cliente_dettagli={"ragioneSociale": "Supermercati Spa", "
                partitaIVA": "11223344556"}
        )
        self.financial_service.registraEntrata(
            categoria_prodotto="VERDURA",
            prodotto_id=None,
            cliente_tipo="Privato",
            importo=30.0,
            data="2025-07-01",
            descrizione="Vendita dettaglio",
            cliente_dettagli={"Nome": "Giuseppe Verdi", "codiceFiscale": "
                VRDGPP80A01H501U"}
        )

        entrate_2026 = self.financial_service.filtraEntrate(anno="2026")
        self.assertEqual(len(entrate_2026), 1)
        self.assertEqual(entrate_2026[0].prezzoTotale, 500.0)

        entrate_prod = self.financial_service.filtraEntrate(prodotto_id=prod
            .idProdotto)
        self.assertEqual(len(entrate_prod), 1)

        aziende = self.financial_service.filtraEntrate(tipo_cliente="Azienda
            ")
        self.assertEqual(len(aziende), 1)
        privati = self.financial_service.filtraEntrate(tipo_cliente="Privato
            ")
        self.assertEqual(len(privati), 1)
```

7.3.3 Persistance tests

- Test di integrazione del livello dati.
- Verifica il corretto salvataggio e ripristino dei dati su file JSON.

```
class TestPersistence(unittest.TestCase):
def setUp(self):
    self.temp_dir = tempfile.mkdtemp()
    self.repo = DataRepository(data_dir=self.temp_dir)
    self.user_manager = GestoreUtenti(self.repo)
    self.product_service = GestoreProdotti(self.repo)
    self.financial_service = GestoreMovimenti(self.repo)

def tearDown(self):
    shutil.rmtree(self.temp_dir)

def test_json_persistence(self):
    self.user_manager.creaManager(
        "m1", "Pass1234", "Mario", "Rossi", "mario@azienda.it", "123", "
            1980-01-01"
    )
    self.product_service.aggiungiCategoria("MIELE", "grammi")
    prod = self.product_service.aggiungiProdotto(
        "Miele Acacia", "Miele biologico", 8.5, "grammi", "MIELE"
    )
    self.financial_service.registraEntrata(
        "MIELE", prod.idProdotto, "Privato", 425.0, "2026-07-22", "
            Vendita miele"
    )

    repo_reload = DataRepository(data_dir=self.temp_dir)

    users = repo_reload.caricaUtenti()
    categories = repo_reload.caricaCategorie()
    products = repo_reload.caricaProdotti()
    movements = repo_reload.caricaMovimenti()

    self.assertEqual(len(users), 1)
    self.assertEqual(users[0].nomeUtente, "m1")
    self.assertEqual(len(categories), 1)
    self.assertEqual(categories[0].nome, "MIELE")
    self.assertEqual(len(products), 1)
    self.assertEqual(products[0].nome, "Miele Acacia")
    self.assertEqual(len(movements), 1)
    self.assertEqual(movements[0].prezzoTotale, 425.0)
```

7.3.4 Views tests

- Test di interfaccia grafica.
- Verifica il caricamento, l'interazione con i controlli e la coerenza delle finestre grafiche.

```
    class TestViews(unittest.TestCase):
    def setUp(self):
        self.temp_dir = tempfile.mkdtemp()
        self.repo = DataRepository(data_dir=self.temp_dir)
        self.auth_service = Autenticatore(self.repo)
        self.user_manager = GestoreUtenti(self.repo)
        self.product_service = GestoreProdotti(self.repo)
        self.financial_service = GestoreMovimenti(self.repo)
        self.report_service = GestoreReport(self.repo)

        self.manager_user = self.user_manager.creaManager(
            username="admin",
            password="Password123",
            nome="Admin",
            cognome="Manager",
            email="admin@azienda.it",
            telefono="1234567890",
            dataNascita="1985-01-01"
        )

        self.dipendente_user = self.user_manager.creaDipendente(
            username="dip1",
            password="Password123",
            nome="Mario",
            cognome="Rossi",
            email="mario.rossi@azienda.it",
            telefono="0987654321",
            dataNascita="1995-05-15"
        )

    def tearDown(self):
        shutil.rmtree(self.temp_dir)

    def test_product_view_columns_and_headers(self):
        view = ProductManagementView(self.product_service)
        headers = [view.table.horizontalHeaderItem(i).text() for i in range(
            view.table.columnCount())]

        expected_headers = [
            "Nome Prodotto", "Categoria / Tipo", "Unit  di Misura", "
                Descrizione", "Prezzo Unitario"
        ]
        self.assertEqual(headers, expected_headers)

    def test_entrate_table_columns(self):
        view = FinancialMovementView(self.financial_service, self.
            product_service, self.manager_user)
        headers = [view.table_entrate.horizontalHeaderItem(i).text() for i
            in range(view.table_entrate.columnCount())]

        expected_headers = [
            "Data", "Prodotto", "Categoria Prodotto", "Cliente",
            "Quantit ", "Unit  di Misura", "Importo Totale (  )", "
                Descrizione", "Allegato PDF"
        ]
        self.assertEqual(headers, expected_headers)

    def test_uscite_table_columns(self):
        view = FinancialMovementView(self.financial_service, self.
            product_service, self.manager_user)
        headers = [view.table_uscite.horizontalHeaderItem(i).text() for i in
```

```
            range(view.table_uscite.columnCount())]

        expected_headers = [
            "Data", "Categoria Spesa", "Fornitore", "Importo Totale (   )",
                "Descrizione", "Allegato PDF"
        ]
        self.assertEqual(headers, expected_headers)

    def test_movement_view_filters(self):
        view = FinancialMovementView(self.financial_service, self.
            product_service, self.manager_user)

        self.assertTrue(hasattr(view, "cb_prod_entrate"))
        self.assertFalse(hasattr(view, "cb_prod_uscite"))
        self.assertEqual(view.cb_prod_entrate.itemText(0), "Tutti i prodotti
            ")

        self.assertTrue(hasattr(view, "cb_spesa_uscite"))
        self.assertFalse(hasattr(view, "cb_spesa_entrate"))
        self.assertEqual(view.cb_spesa_uscite.itemText(0), "Tutti i tipi di
            spesa")
        spese_options = [view.cb_spesa_uscite.itemText(i) for i in range(1,
            view.cb_spesa_uscite.count())]
        expected_spese = [e.value for e in TipoUscita]
        self.assertEqual(spese_options, expected_spese)

        self.assertTrue(hasattr(view, "cb_cliente_entrate"))
        self.assertFalse(hasattr(view, "cb_cliente_uscite"))
        client_options = [view.cb_cliente_entrate.itemText(i) for i in range
            (view.cb_cliente_entrate.count())]
        self.assertEqual(client_options, ["Tutti i clienti", "Azienda", "
            Privato"])

    def test_main_window_tabs_for_manager(self):
        window = MainWindow(self.manager_user, self.repo, self.auth_service)
        tab_titles = [window.tabs.tabText(i) for i in range(window.tabs.
            count())]

        expected_tabs = [
            "Gestione Utenti",
            "Catalogo Prodotti Agricoli",
            "Movimenti",
            "Guadagno Aziendale"
        ]
        self.assertEqual(tab_titles, expected_tabs)

    def test_main_window_tabs_for_dipendente(self):
        window = MainWindow(self.dipendente_user, self.repo, self.
            auth_service)
        tab_titles = [window.tabs.tabText(i) for i in range(window.tabs.
            count())]

        expected_tabs = [
            "Gestione Utenti",
            "Catalogo Prodotti Agricoli",
            "Movimenti"
        ]
        self.assertEqual(tab_titles, expected_tabs)
        self.assertNotIn("Guadagno Aziendale", tab_titles)

    def test_user_management_view_permissions(self):
        view_mgr = UserManagementView(self.manager_user, self.user_manager,
```

```
            self.auth_service)
        self.assertTrue(hasattr(view_mgr, "users_table"))
        self.assertEqual(view_mgr.users_table.columnCount(), 6)

        headers = [view_mgr.users_table.horizontalHeaderItem(i).text() for i
             in range(view_mgr.users_table.columnCount())]
        self.assertEqual(headers, ["Username", "Ruolo", "Nome", "Cognome", "
            Email", "Ultimo Login"])

        view_dip = UserManagementView(self.dipendente_user, self.
            user_manager, self.auth_service)
        self.assertFalse(hasattr(view_dip, "users_table"))

    def test_report_view_elements(self):
        view = ReportView(self.report_service)
        self.assertTrue(hasattr(view, "cb_anno"))
        self.assertTrue(hasattr(view, "report_display"))

        curr_year = QDate.currentDate().year()
        self.assertEqual(view.cb_anno.itemText(0), str(curr_year))
        self.assertIn("REPORT GUADAGNO AZIENDALE", view.report_display.
            toPlainText())

    def test_login_dialog_modes(self):
        dialog = LoginDialog(self.auth_service, self.user_manager)
        self.assertEqual(dialog.stacked_widget.currentWidget(), dialog.
            login_widget)

        empty_dir = tempfile.mkdtemp()
        try:
            empty_repo = DataRepository(data_dir=empty_dir)
            empty_um = GestoreUtenti(empty_repo)
            empty_auth = Autenticatore(empty_repo)
            dialog_first = LoginDialog(empty_auth, empty_um)
            self.assertEqual(dialog_first.stacked_widget.currentWidget(),
                dialog_first.first_manager_widget)
        finally:
            shutil.rmtree(empty_dir)
```